

AI/BigData 인텐시브 과정

강사장철원

Section
코스소개

DAY1

머신러닝
기초 개념

DAY2

지도학습

DAY3

DAY4

서비스
개발

DAY5

앙상블 학습

DAY6

비지도학습

DAY7

시계열분석

DAY8

DAY9

프로젝트

DAY10

□ 사이킷런 라이브러리 소개

□ kNN

□ 회귀분석

□ 로지스틱 회귀분석

□ 나이브 베이즈

□ 의사결정나무

□ 서포트벡터머신

AI/BigData 인텐시브 과정

Section 1. 라이브러리 소개

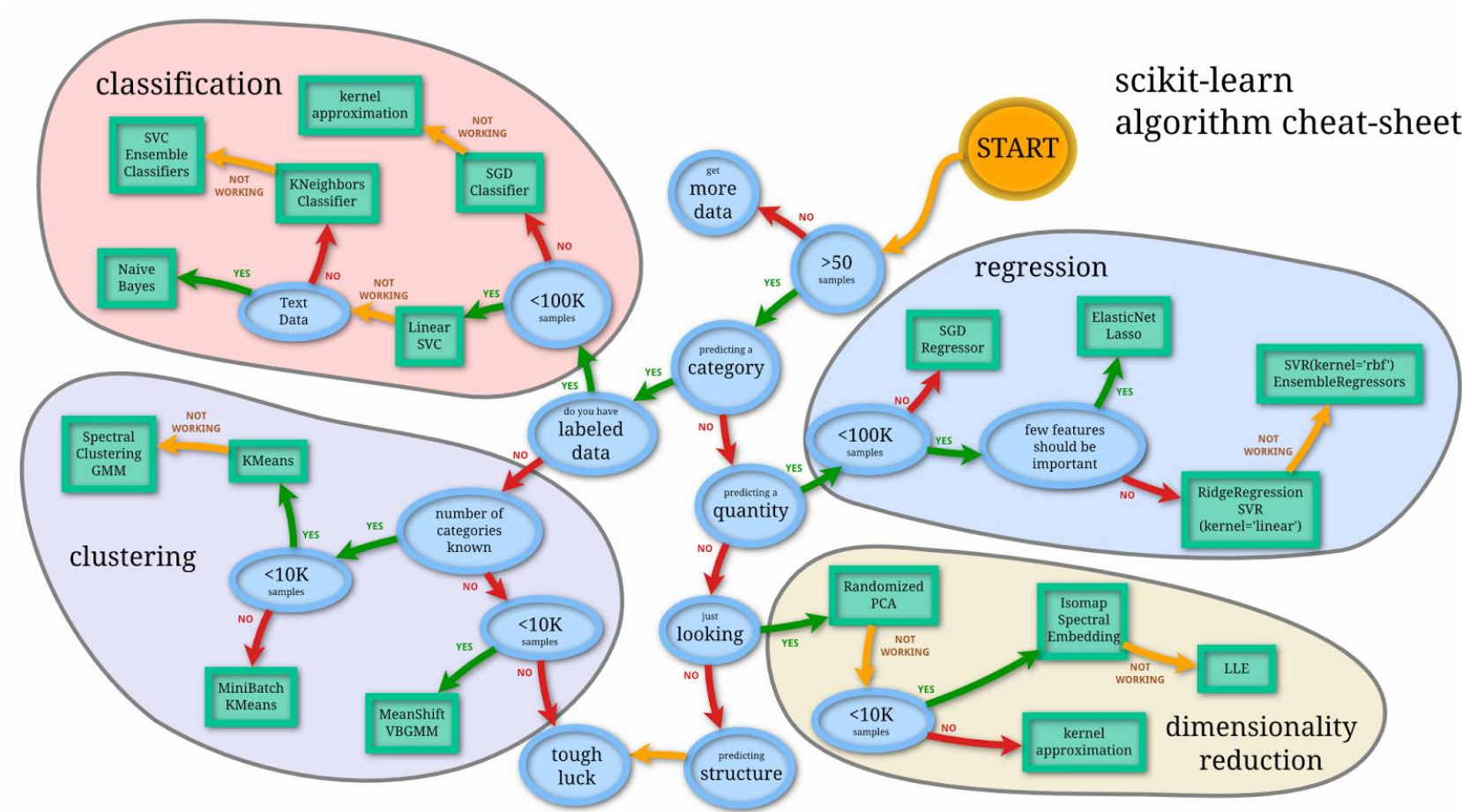
Section 1-1. 사이킷런 라이브러리

사이킷런(scikit-learn)

파이썬을 활용해 머신러닝을 사용할 수 있게 도와주는 라이브러리



사이킷런(scikit-learn)



사이킷런(scikit-learn)

알고리즘 별로 편리하게 사용할 수 있도록 제공

메소드	기능
fit	모형 적합
predict	예측

AI/BigData 인텐시브 과정

Section 2. K 최근접 이웃

Section 2-1. k 최근접 이웃의 기초 개념

k 최근접 이웃 알고리즘(KNN, K Nearest Neighbor)

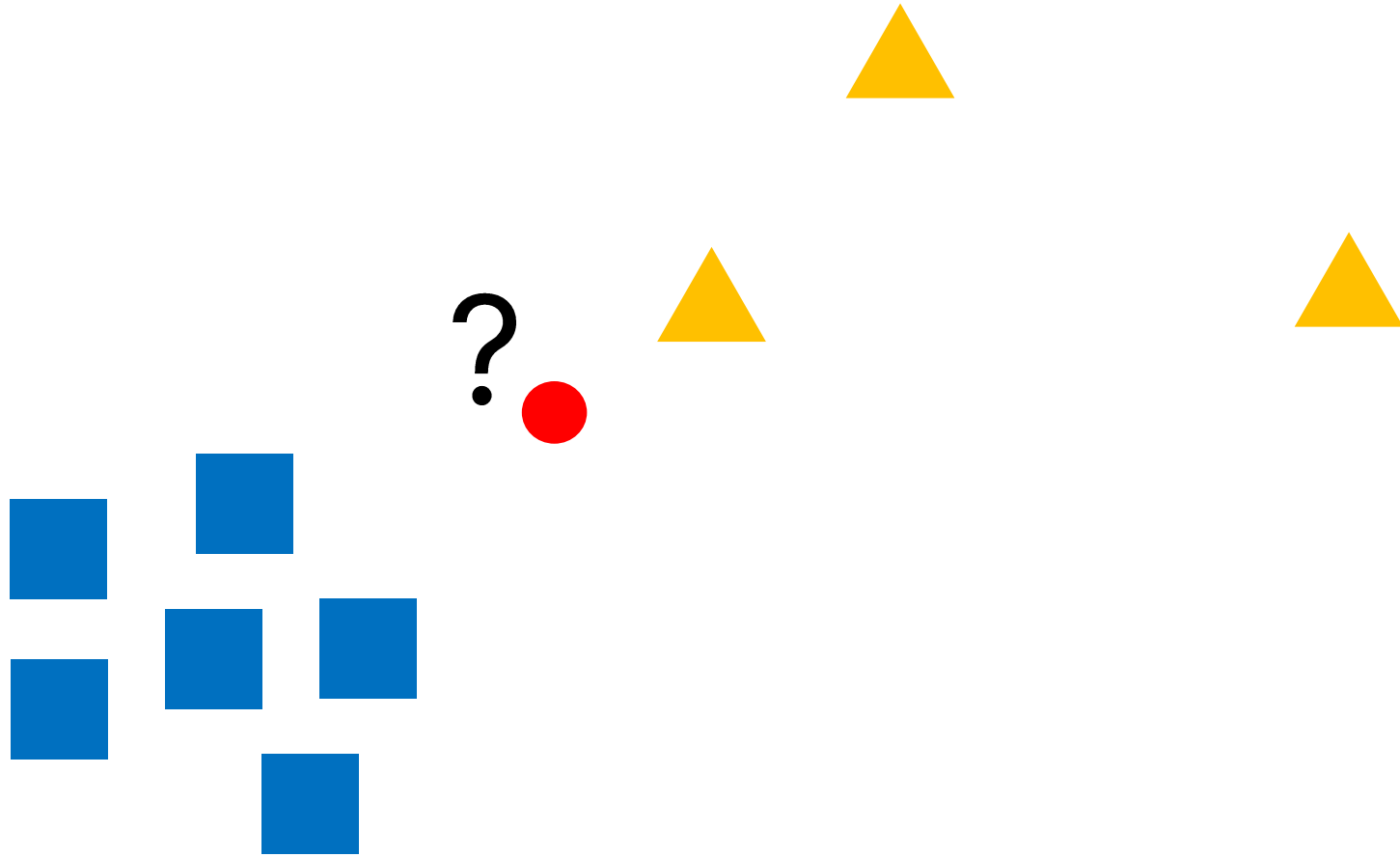
K Nearest Neighbor

K

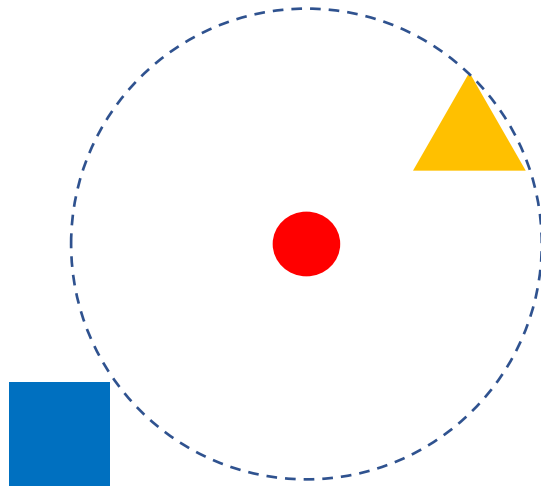
최근접

이웃

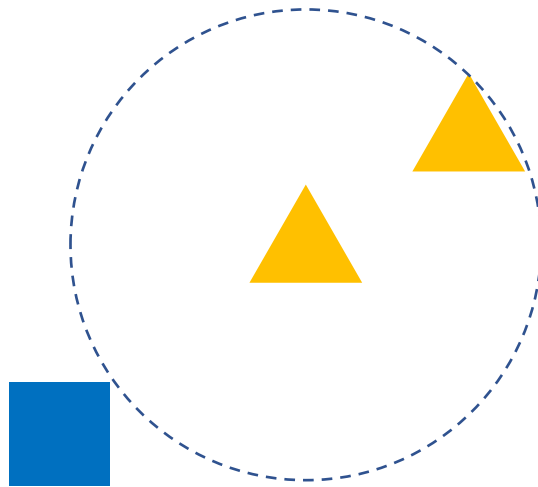
k 최근접 이웃 알고리즘(KNN, K Nearest Neighbor)



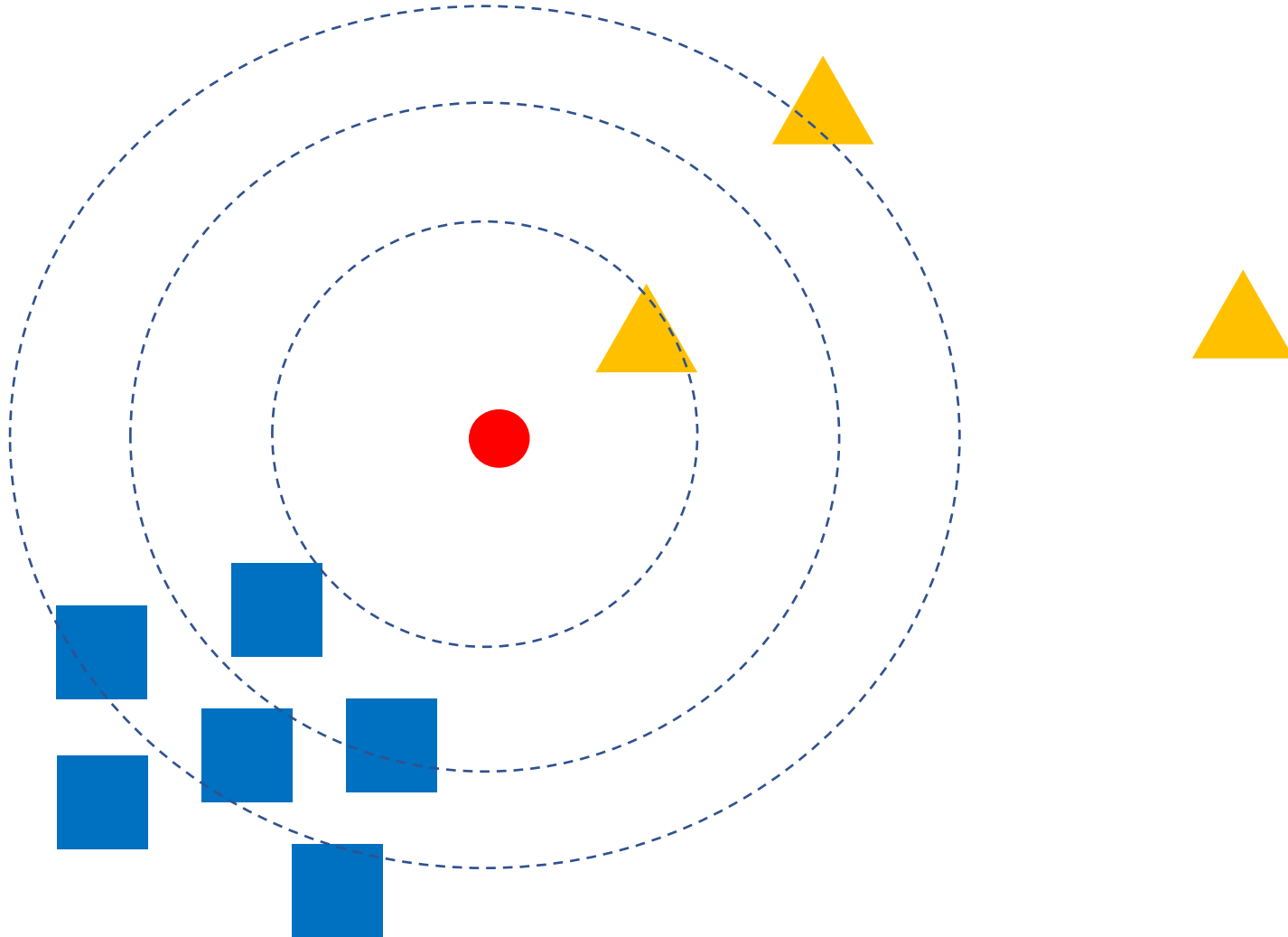
k 최근접 이웃 알고리즘(KNN, K Nearest Neighbor)



k 최근접 이웃 알고리즘(KNN, K Nearest Neighbor)



k 최근접 이웃 알고리즘(KNN, K Nearest Neighbor)



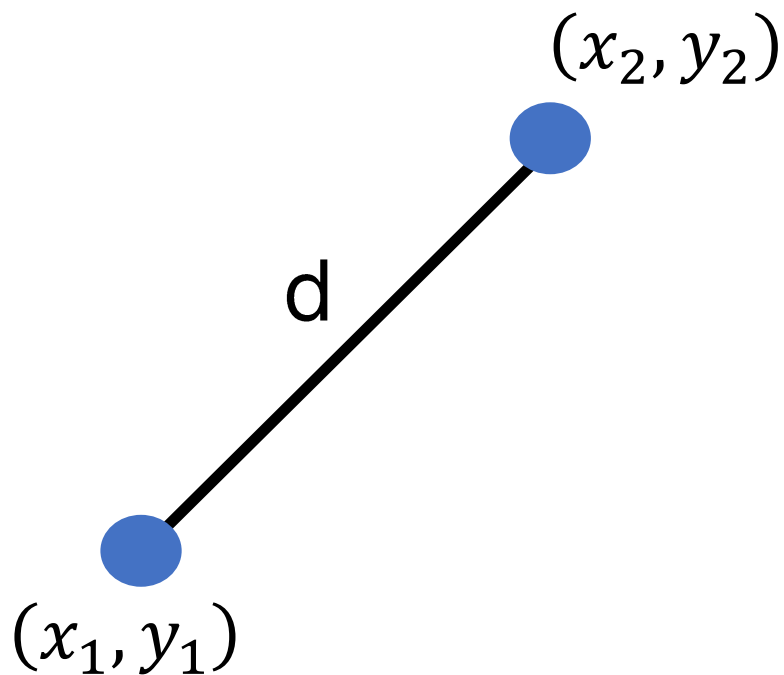
AI/BigData 인텐시브 과정

Section 2. K 최근접 이웃

Section 2-2. 유클리디언 거리

유클리디언 거리

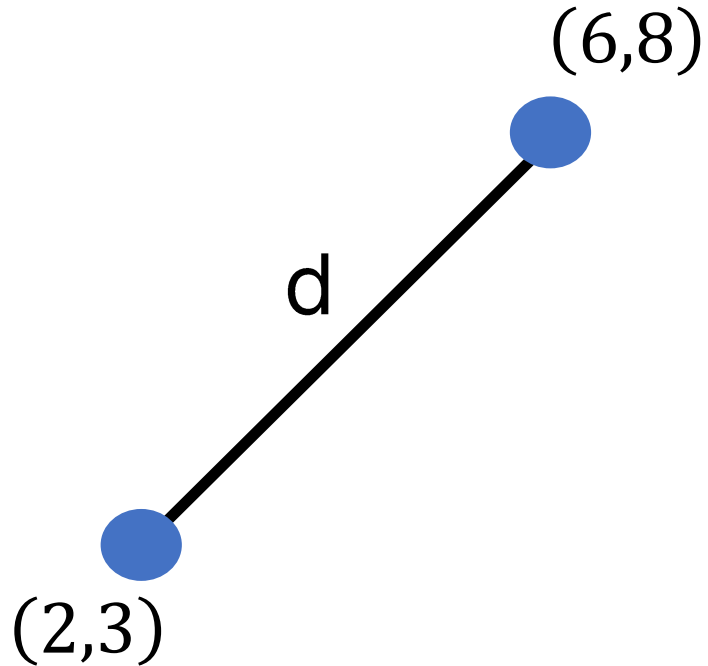
데이터 포인트 간 거리 측정 기준은 기본적으로 유클리디언 거리



유클리디언 거리

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

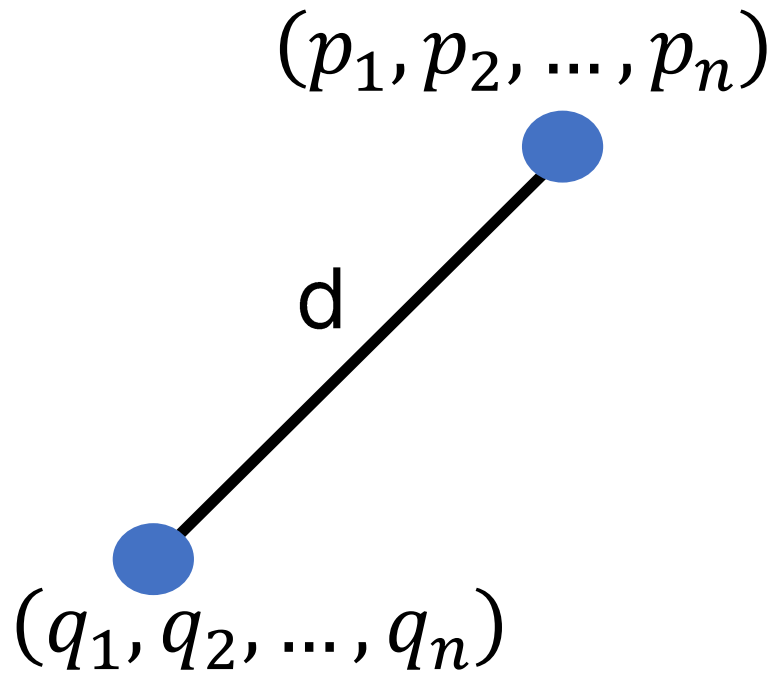
유클리디언 거리(2차원)



유클리디언 거리

$$\begin{aligned} d &= \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2} \\ &= \sqrt{(6 - 2)^2 + (8 - 3)^2} \\ &= \sqrt{16 + 25} = \sqrt{41} \\ &= 6.4 \end{aligned}$$

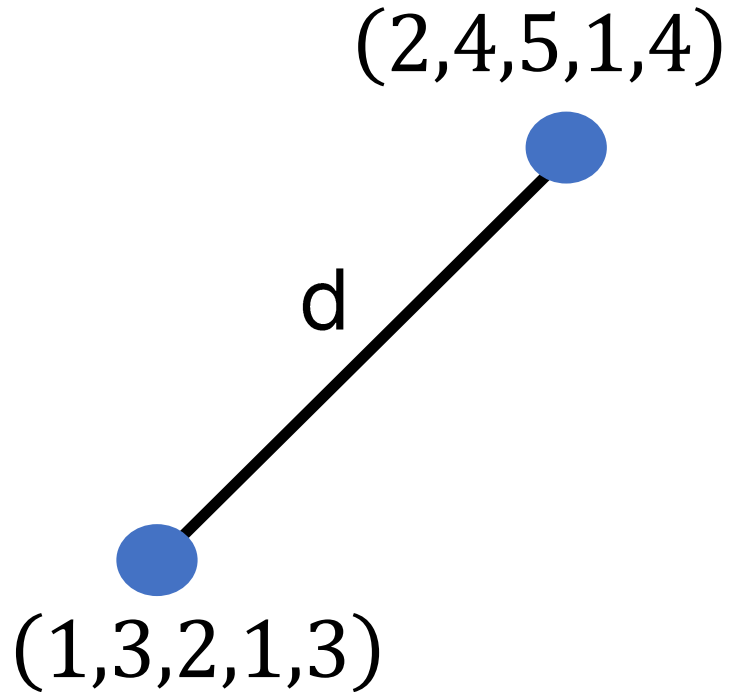
유클리디언 거리(n차원)



유클리디언 거리

$$d = \sqrt{(p_1 - q_1)^2 + (p_2 - q_2)^2 + \dots + (p_n - q_n)^2}$$

유클리디언 거리(n차원)



유클리디언 거리

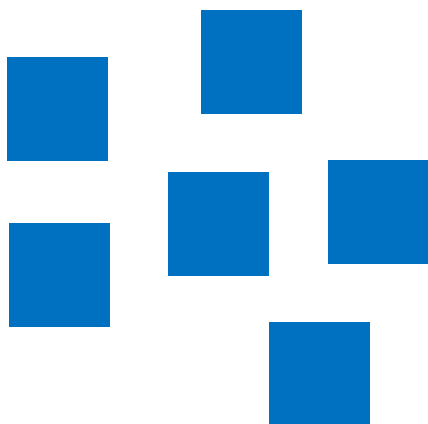
$$\begin{aligned} d &= \sqrt{(p_1 - q_1)^2 + (p_2 - q_2)^2 + \cdots + (p_n - q_n)^2} \\ &= \sqrt{(2 - 1)^2 + (4 - 3)^2 + (5 - 2)^2 + (1 - 1)^2 + (4 - 3)^2} \\ &= \sqrt{1^2 + 1^2 + 3^2 + 0^2 + 1^2} \\ &= \sqrt{12} \\ &= 3.46 \end{aligned}$$

AI/BigData 인텐시브 과정

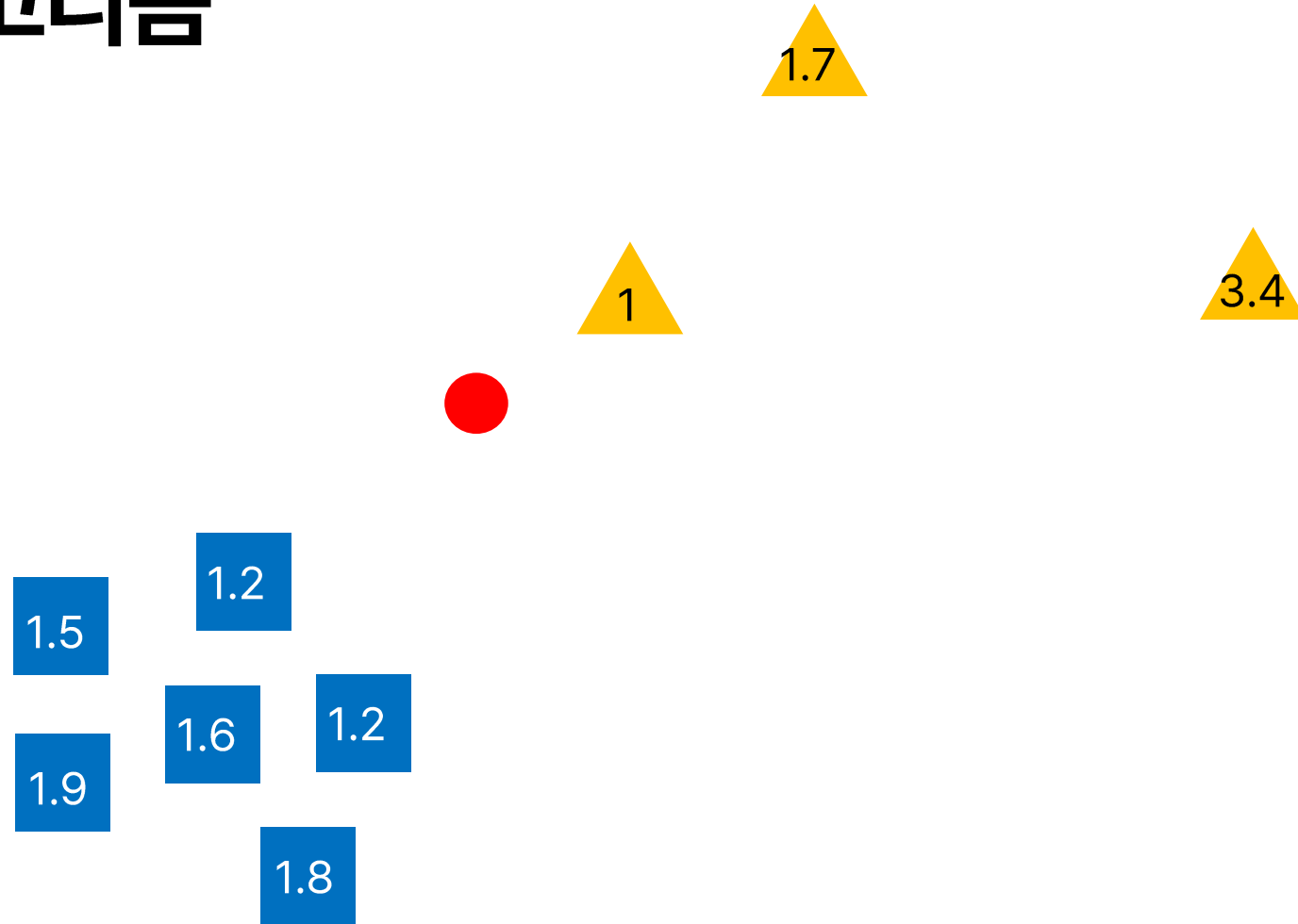
Section 2. K 최근접 이웃

Section 2-3. k 최근접 이웃 알고리즘

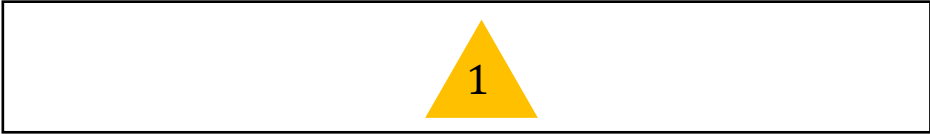
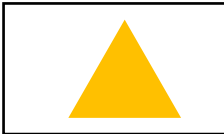
k 최근접 이웃 알고리즘



k 최근접 이웃 알고리즘



k 최근접 이웃 알고리즘

1-Nearest Neighbor		
3-Nearest Neighbor		
5-Nearest Neighbor		
7-Nearest Neighbor		

k 최근접 이웃 알고리즘

- ✓ 가장 가까이 있는 데이터 포인트 라벨에 속한다고 보는 방법
- ✓ 가까이 있는 데이터 1개를 보면 1-최근접 이웃
- ✓ 가까이 있는 데이터 n개를 보면 n-최근접 이웃

AI/BigData 인텐시브 과정

Section 2. K 최근접 이웃

Section 2-4. k 최근접 이웃 분류

k 최근접 이웃 알고리즘

```
[1]: import pandas as pd  
df = pd.read_csv("../data/iris.csv")
```

```
[2]: df
```

```
[2]:
```

	sepal_length	sepal_width	petal_length	petal_width	class
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0
...	...	...	...	...	...
145	6.7	3.0	5.2	2.3	2
146	6.3	2.5	5.0	1.9	2
147	6.5	3.0	5.2	2.0	2
148	6.2	3.4	5.4	2.3	2
149	5.9	3.0	5.1	1.8	2

150 rows × 5 columns

k최근접 이웃분류

[illegible]

[5]:	sepal_length	sepal_width	petal_length	petal_width
0	5.1	3.5	1.4	0.2
1	4.9	3.0	1.4	0.2
2	4.7	3.2	1.3	0.2
3	4.6	3.1	1.5	0.2
4	5.0	3.6	1.4	0.2
...	...	...	...	...
145	6.7	3.0	5.2	2.3
146	6.3	2.5	5.0	1.9
147	6.5	3.0	5.2	2.0
148	6.2	3.4	5.4	2.3
149	5.9	3.0	5.1	1.8

150 rows x 4 columns

k 최근접 이웃 알고리즘

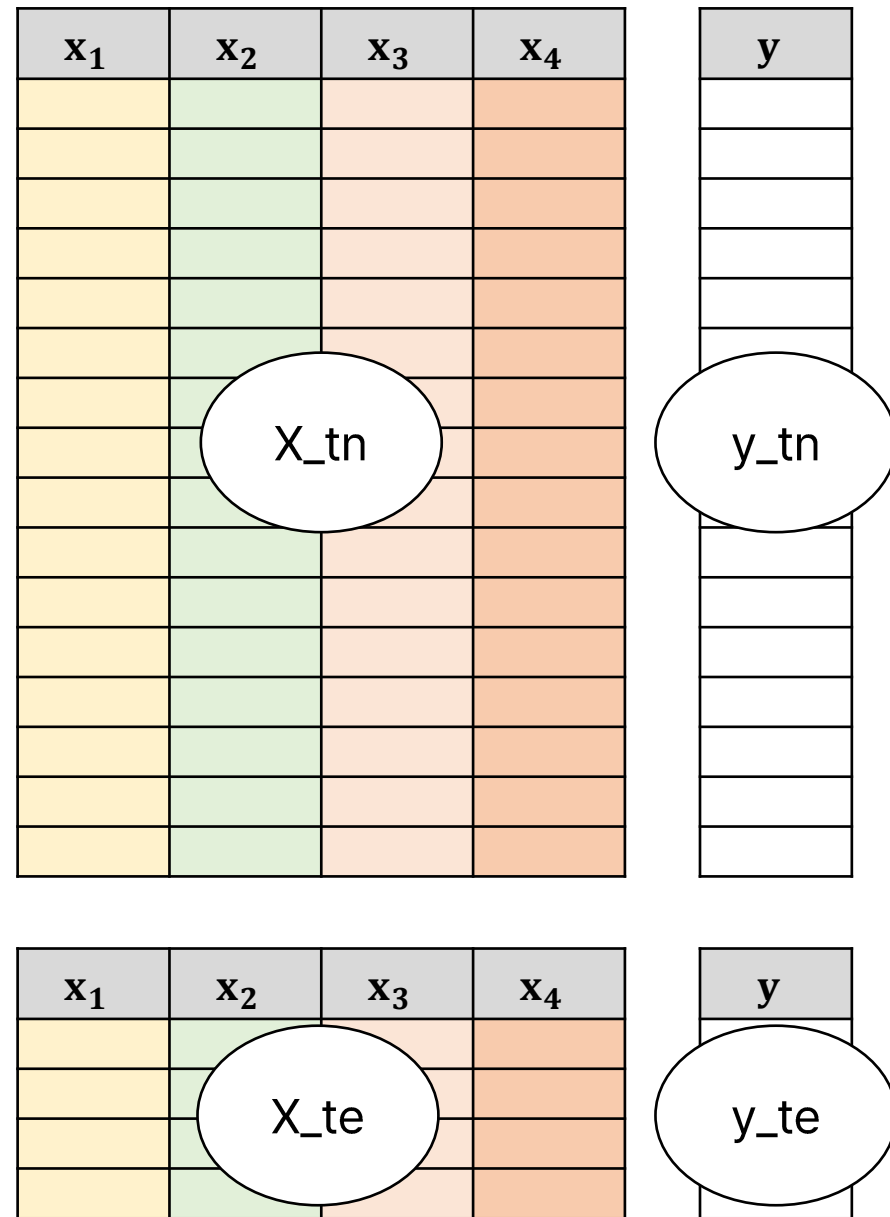
```
In [6]: ▶ 1 # 트레이닝/테스트 데이터 분할
          2 from sklearn.model_selection import train_test_split
          3 X_tn, X_te, y_tn, y_te=train_test_split(X,y,random_state=0)
```

```
In [7]: ▶ 1 X_tn
```

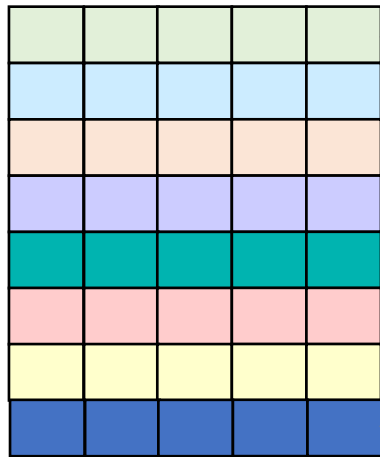
Out[7]:

	sepal_length	sepal_width	petal_length	petal_width
61	5.9	3.0	4.2	1.5
92	5.8	2.6	4.0	1.2
112	6.8	3.0	5.5	2.1
2	4.7	3.2	1.3	0.2
141	6.9	3.1	5.1	2.3
...	...	...	...	...
9	4.9	3.1	1.5	0.1
103	6.3	2.9	5.6	1.8
67	5.8	2.7	4.1	1.0
117	7.7	3.8	6.7	2.2
47	4.6	3.2	1.4	0.2

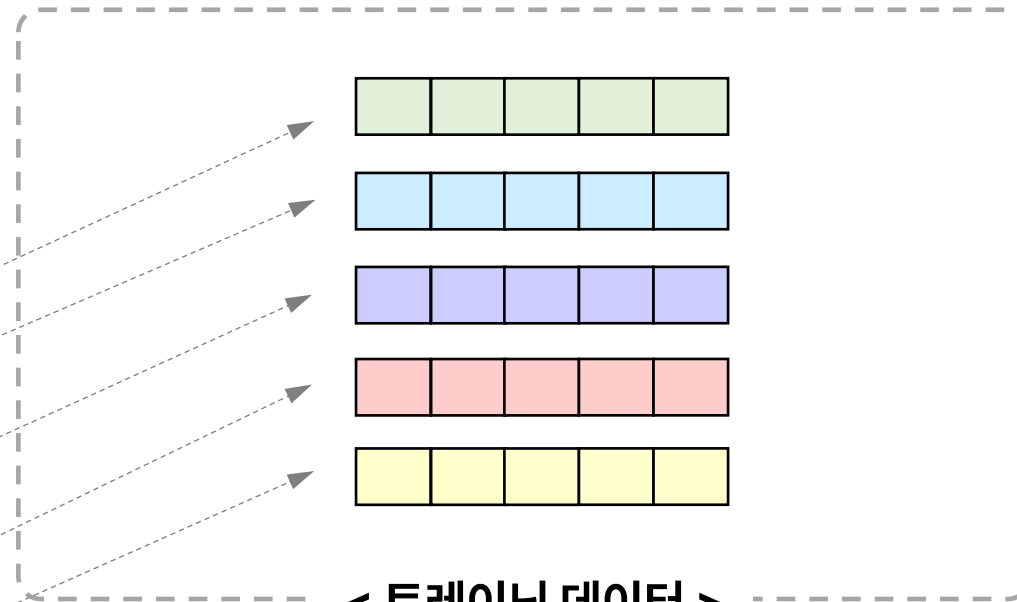
112 rows x 4 columns



tran_test_split



< 전체 데이터 >



< 트레이닝 데이터 >



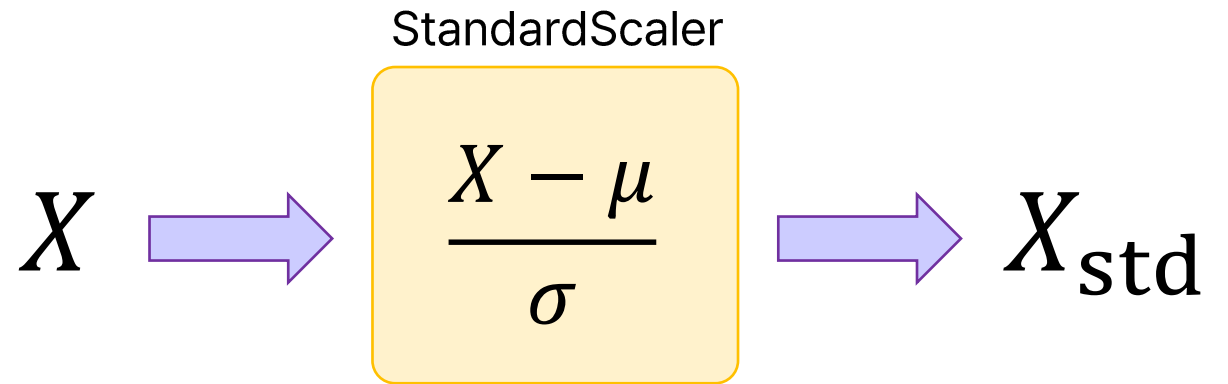
< 테스트 데이터 >

k 최근접 이웃 알고리즘

```
•[8]: #Standardization
from sklearn.preprocessing import StandardScaler
std_scale = StandardScaler() → StandardScaler 객체 생성
std_scale.fit(X_tn)
X_tn_std = std_scale.transform(X_tn)
X_te_std = std_scale.transform(X_te)
```

```
[9]: X_tn_std
```

```
-8.01118523e-01],
[-5.60984968e-01,  1.49274053e+00, -1.28103155e+00,
-1.31530348e+00],
[-1.02212490e+00, -2.42210516e+00, -1.65358660e-01,
-2.86933568e-01],
[ 7.07149859e-01, -1.19254753e-01,  9.50314227e-01,
 7.41436341e-01],
[ 9.37719827e-01,  5.71600368e-01,  1.06188152e+00,
 1.64126001e+00],
[ 1.30724937e-01, -1.96153508e+00,  6.71396005e-01,
 3.55797625e-01],
[ 9.37719827e-01, -1.27067995e+00,  1.11766516e+00,
 7.41436341e-01],
[-3.30414999e-01, -1.27067995e+00,  5.77759173e-02,
-1.58387330e-01],
[ 2.09056967e+00, -1.19254753e-01,  1.28501609e+00,
 1.38416753e+00],
[ 4.76579890e-01,  5.71600368e-01,  5.04045072e-01,
```



k 최근접 이웃 알고리즘

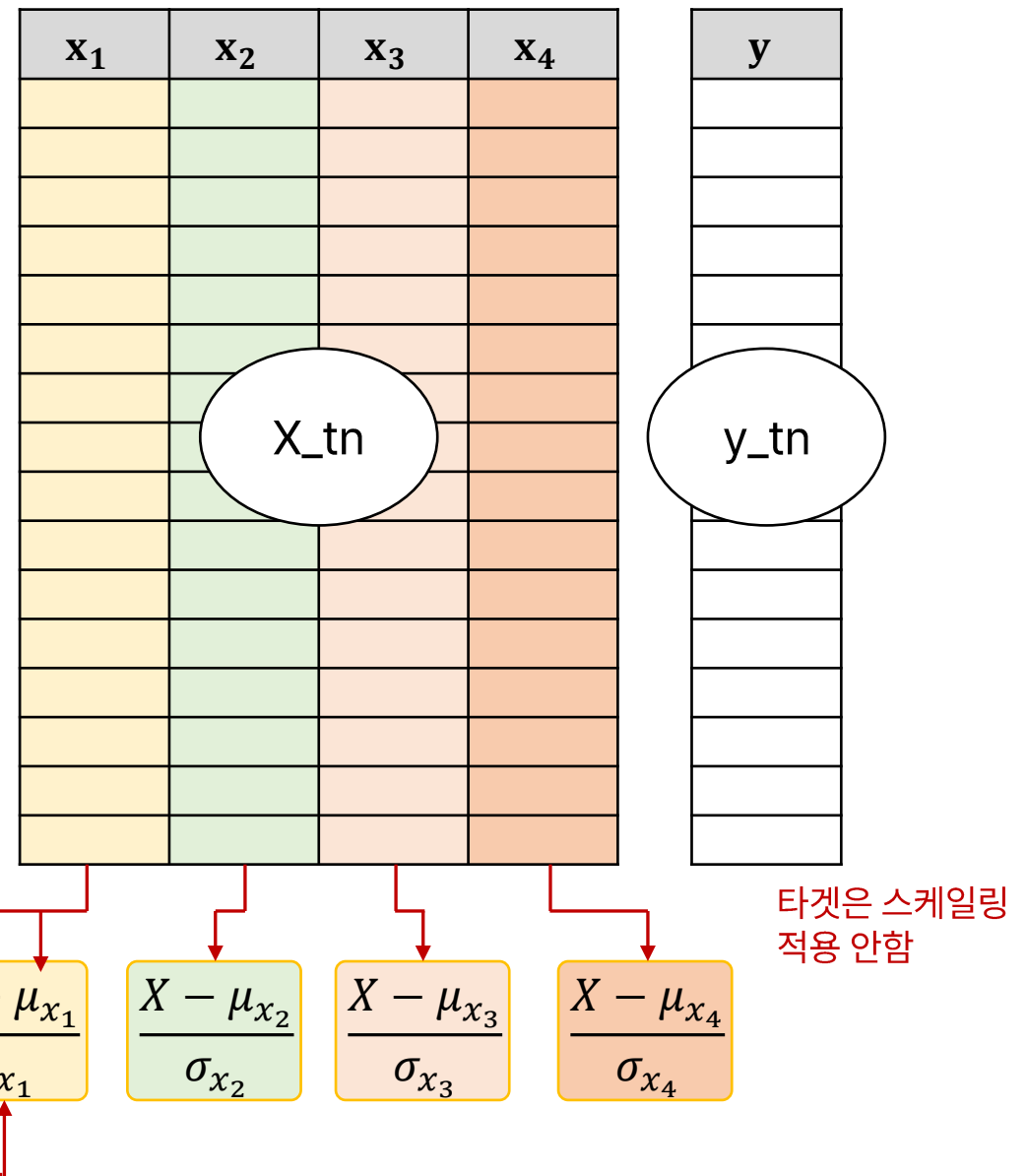
•[8]: #Standardization

```
from sklearn.preprocessing import StandardScaler
std_scale = StandardScaler()
std_scale.fit(X_tn)
X_tn_std = std_scale.transform(X_tn)
X_te_std = std_scale.transform(X_te)
```

→ 피처별 평균, 표준편차 계산

[9]: X_tn_std

```
-8.01118523e-01],
[-5.60984968e-01,  1.49274053e+00, -1.28103155e+00,
-1.31530348e+00],
[-1.02212490e+00, -2.42210516e+00, -1.65358660e-01,
-2.86933568e-01],
[ 7.07149859e-01, -1.19254753e-01,  9.50314227e-01,
 7.41436341e-01],
[ 9.37719827e-01,  5.71600368e-01,  1.06188152e+00,
 1.64126001e+00],
[ 1.30724937e-01, -1.96153508e+00,  6.71396005e-01,
 3.55797625e-01],
[ 9.37719827e-01, -1.27067995e+00,  1.11766516e+00,
 7.41436341e-01],
[-3.30414999e-01, -1.27067995e+00,  5.77759173e-02,
-1.58387330e-01],
[ 2.09056967e+00, -1.19254753e-01,  1.28501609e+00,
 1.38416753e+00],
[ 4.76579890e-01,  5.71600368e-01,  5.04045072e-01,
```



k 최근접 이웃 알고리즘

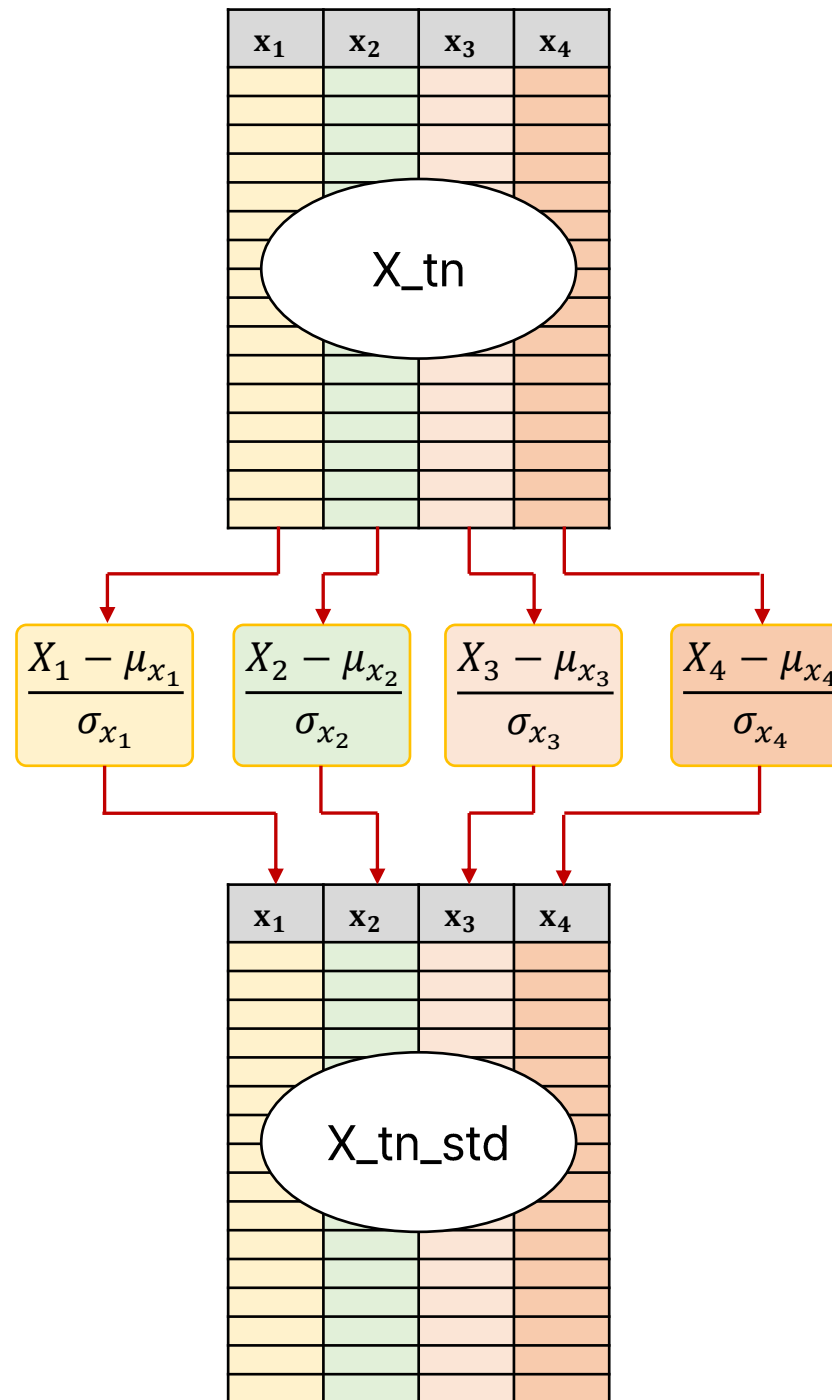
• [8]: #Standardization

```
from sklearn.preprocessing import StandardScaler
std_scale = StandardScaler()
std_scale.fit(X_tn)
X_tn_std = std_scale.transform(X_tn)
X_te_std = std_scale.transform(X_te)
```

→ 트레이닝 데이터 변환

[9]: X_tn_std

```
-8.01118523e-01],
[-5.60984968e-01,  1.49274053e+00, -1.28103155e+00,
-1.31530348e+00],
[-1.02212490e+00, -2.42210516e+00, -1.65358660e-01,
-2.86933568e-01],
[ 7.07149859e-01, -1.19254753e-01,  9.50314227e-01,
 7.41436341e-01],
[ 9.37719827e-01,  5.71600368e-01,  1.06188152e+00,
 1.64126001e+00],
[ 1.30724937e-01, -1.96153508e+00,  6.71396005e-01,
 3.55797625e-01],
[ 9.37719827e-01, -1.27067995e+00,  1.11766516e+00,
 7.41436341e-01],
[-3.30414999e-01, -1.27067995e+00,  5.77759173e-02,
-1.58387330e-01],
[ 2.09056967e+00, -1.19254753e-01,  1.28501609e+00,
 1.38416753e+00],
[ 4.76579890e-01,  5.71600368e-01,  5.04045072e-01,
```



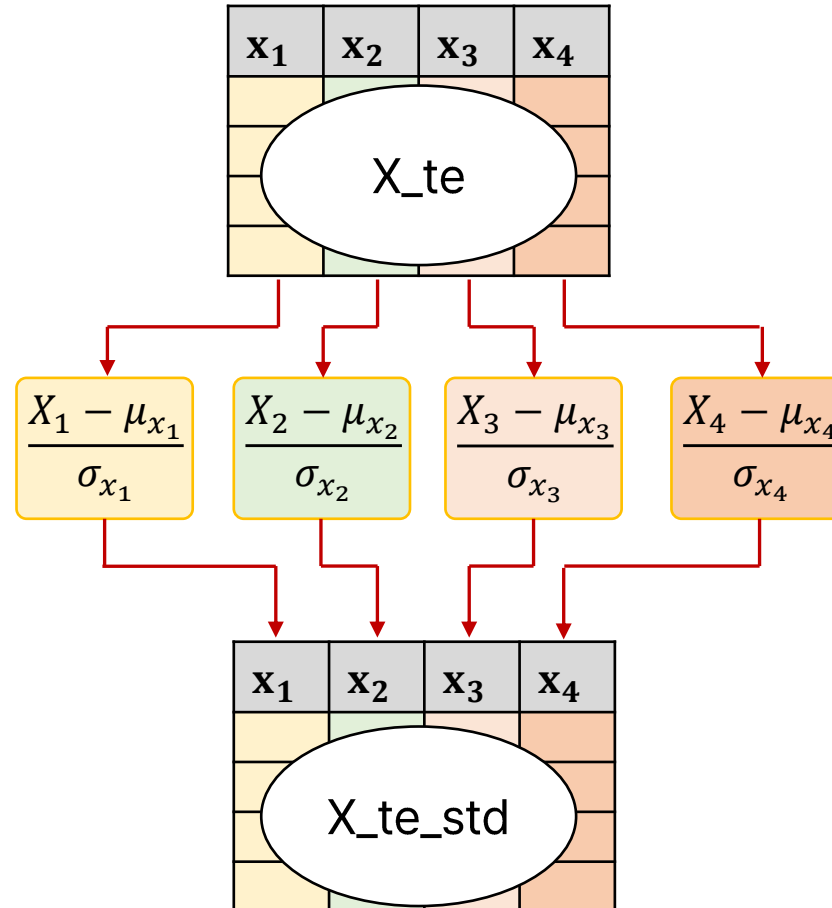
k 최근접 이웃 알고리즘

• [8]: `#Standardization`

```
from sklearn.preprocessing import StandardScaler
std_scale = StandardScaler()
std_scale.fit(X_tn)
X_tn_std = std_scale.transform(X_tn)
X_te_std = std_scale.transform(X_te) → 테스트 데이터 변환
```

[9]: `X_tn_std`

```
-8.01118523e-01],
[-5.60984968e-01,  1.49274053e+00, -1.28103155e+00,
-1.31530348e+00],
[-1.02212490e+00, -2.42210516e+00, -1.65358660e-01,
-2.86933568e-01],
[ 7.07149859e-01, -1.19254753e-01,  9.50314227e-01,
 7.41436341e-01],
[ 9.37719827e-01,  5.71600368e-01,  1.06188152e+00,
 1.64126001e+00],
[ 1.30724937e-01, -1.96153508e+00,  6.71396005e-01,
 3.55797625e-01],
[ 9.37719827e-01, -1.27067995e+00,  1.11766516e+00,
 7.41436341e-01],
[-3.30414999e-01, -1.27067995e+00,  5.77759173e-02,
-1.58387330e-01],
[ 2.09056967e+00, -1.19254753e-01,  1.28501609e+00,
 1.38416753e+00],
[ 4.76579890e-01,  5.71600368e-01,  5.04045072e-01,
```



k 최근접 이웃 알고리즘

[10]: # 학습

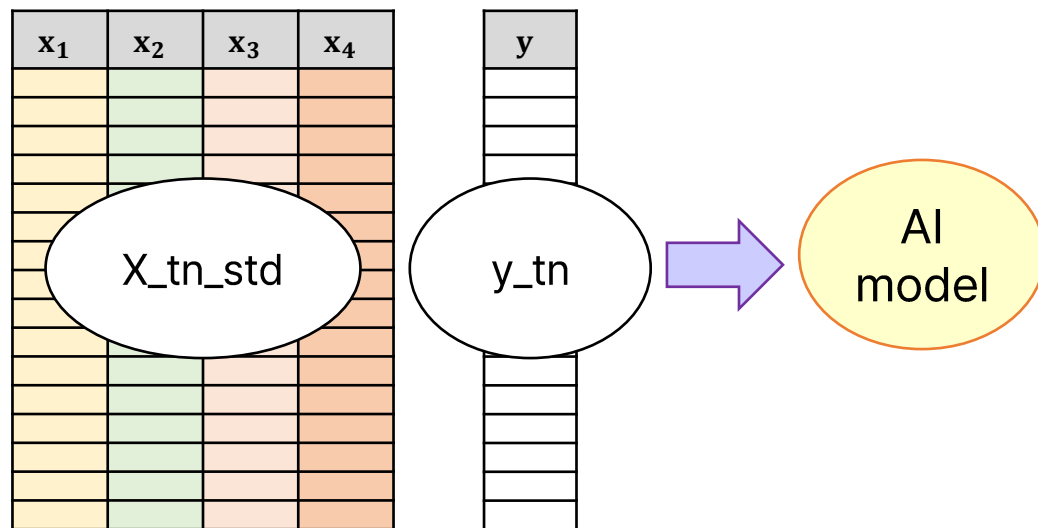
```
from sklearn.neighbors import KNeighborsClassifier  
knn_model = KNeighborsClassifier(n_neighbors=2)  
knn_model.fit(X_tn_std, y_tn)
```

[10]:

▼ KNeighborsClassifier ⓘ ?

▼ Parameters

📄	n_neighbors	2
📄	weights	'uniform'
📄	algorithm	'auto'
📄	leaf_size	30
📄	p	2
📄	metric	'minkowski'
📄	metric_params	None
📄	n_jobs	None



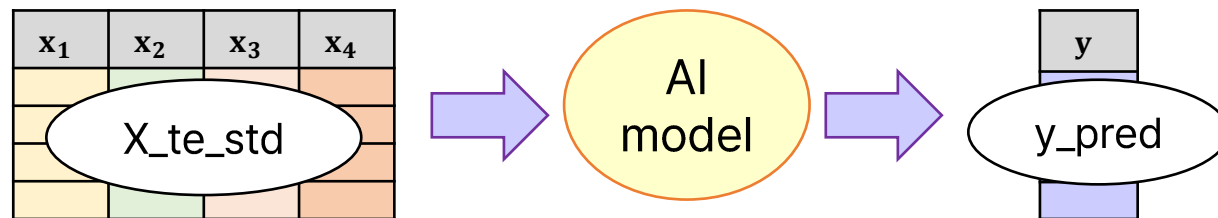
k 최근접 이웃 알고리즘

```
# 예측
```

```
y_pred = knn_model.predict(X_te_std)
```

```
print(y_pred)
```

```
[2 1 0 2 0 2 0 1 1 1 1 1 1 1 0 1 1 0 0 2 1 0 0 2 0 0 1 1 0 2 1 0 2 2 1 0  
2]
```



k 최근접 이웃 알고리즘

[Install](#) [User Guide](#) [API](#) [Examples](#) [Community](#) [More ▾](#)[Prev](#)[Up](#)[Next](#)

scikit-learn 1.3.1

[Other versions](#)

Please [cite us](#) if you use the software.

[sklearn.neighbors.KNeighborsClassifier](#)[KNeighborsClassifier](#)[Examples using](#)[sklearn.neighbors.KNeighborsClass:](#)

sklearn.neighbors.KNeighborsClassifier

```
class sklearn.neighbors.KNeighborsClassifier(n_neighbors=5, *, weights='uniform', algorithm='auto', leaf_size=30, p=2, metric='minkowski', metric_params=None, n_jobs=None)
```

[\[source\]](#)

Classifier implementing the k-nearest neighbors vote.

Read more in the [User Guide](#).

Parameters:

n_neighbors : int, default=5

Number of neighbors to use by default for [kneighbors](#) queries.

weights : {'uniform', 'distance'}, callable or None, default='uniform'

Weight function used in prediction. Possible values:

- 'uniform': uniform weights. All points in each neighborhood are weighted equally.
- 'distance': weight points by the inverse of their distance. In this case, closer neighbors of a query point will have a greater influence than neighbors which are further away.
- [callable]: a user-defined function which accepts an array of distances, and returns an array of the same shape containing the weights.

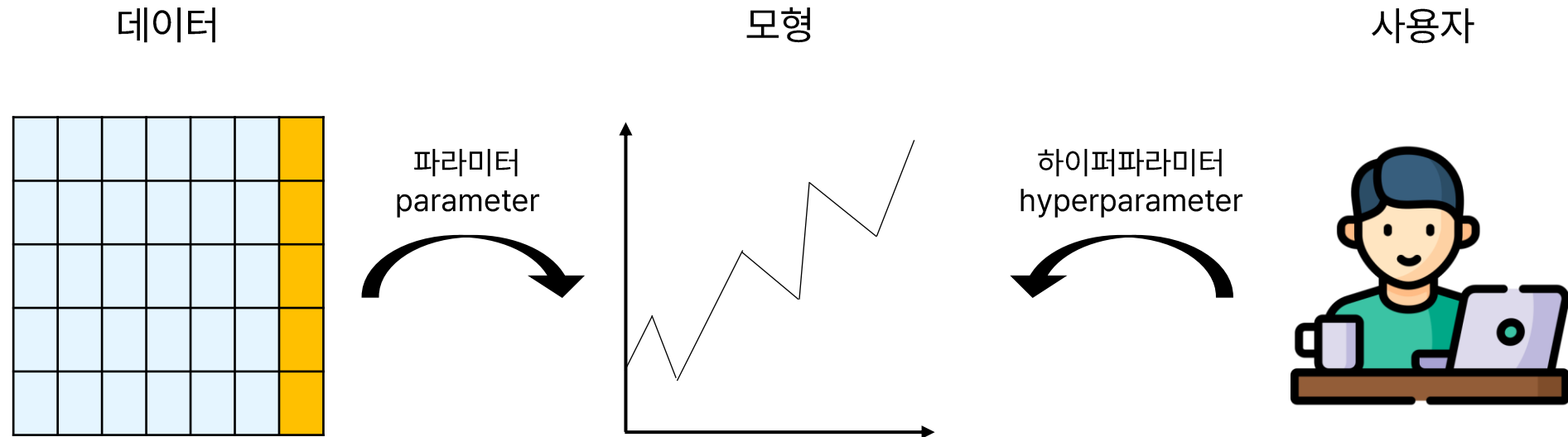
Refer to the example entitled [Nearest Neighbors Classification](#) showing the impact of the `weights` parameter on the decision boundary.

algorithm : {'auto', 'ball_tree', 'kd_tree', 'brute'}, default='auto'

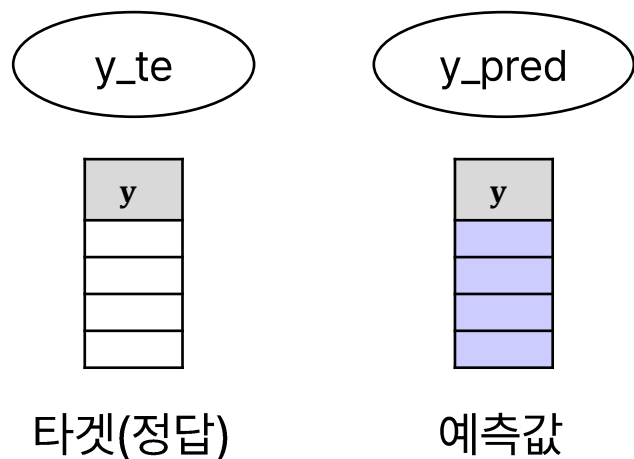
Algorithm used to compute the nearest neighbors:

- 'ball_tree' will use [BallTree](#)
- 'kd_tree' will use [KDTree](#)
- 'brute' will use a brute-force search.

파라미터(parameter) vs 하이퍼 파라미터(hyper parameter)



k 최근접 이웃 알고리즘



```
# 정확도(accuracy)
from sklearn.metrics import accuracy_score
accuracy = accuracy_score(y_te, y_pred)
print(accuracy)
```

0.9473684210526315

```
# 정밀도(precision)
from sklearn.metrics import precision_score
precision = precision_score(y_te, y_pred, average='macro')
print(precision)
```

0.9421296296296297

```
# 리콜
from sklearn.metrics import recall_score
recall = recall_score(y_te, y_pred, average='macro')
print(recall)
```

0.9421296296296297

```
# F1 스코어
from sklearn.metrics import f1_score
f1 = f1_score(y_te, y_pred, average='macro')
print(f1)
```

0.9421296296296297

k 최근접 이웃 알고리즘

```
# confusion matrix 확인
from sklearn.metrics import confusion_matrix
conf_matrix = confusion_matrix(y_te, y_pred)
print(conf_matrix)
```

실제

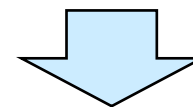
예측

```
[[13  0  0]
 [ 0 15  1]
 [ 0  1  8]]
```

```
# 분류 레포트 확인
from sklearn.metrics import classification_report
class_report = classification_report(y_te, y_pred)
print(class_report)
```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	13
1	0.94	0.94	0.94	16
2	0.89	0.89	0.89	9
accuracy			0.95	38
macro avg	0.94	0.94	0.94	38
weighted avg	0.95	0.95	0.95	38

		예측		
		0	1	2
실제	0	13	0	0
	1	0	15	1
	2	0	1	8



		예측	
		0	not 0
실제	0	13	0+0
	not 0	0+0	15+1+1+8=25

AI/BigData 인텐시브 과정

Section 2. K 최근접 이웃

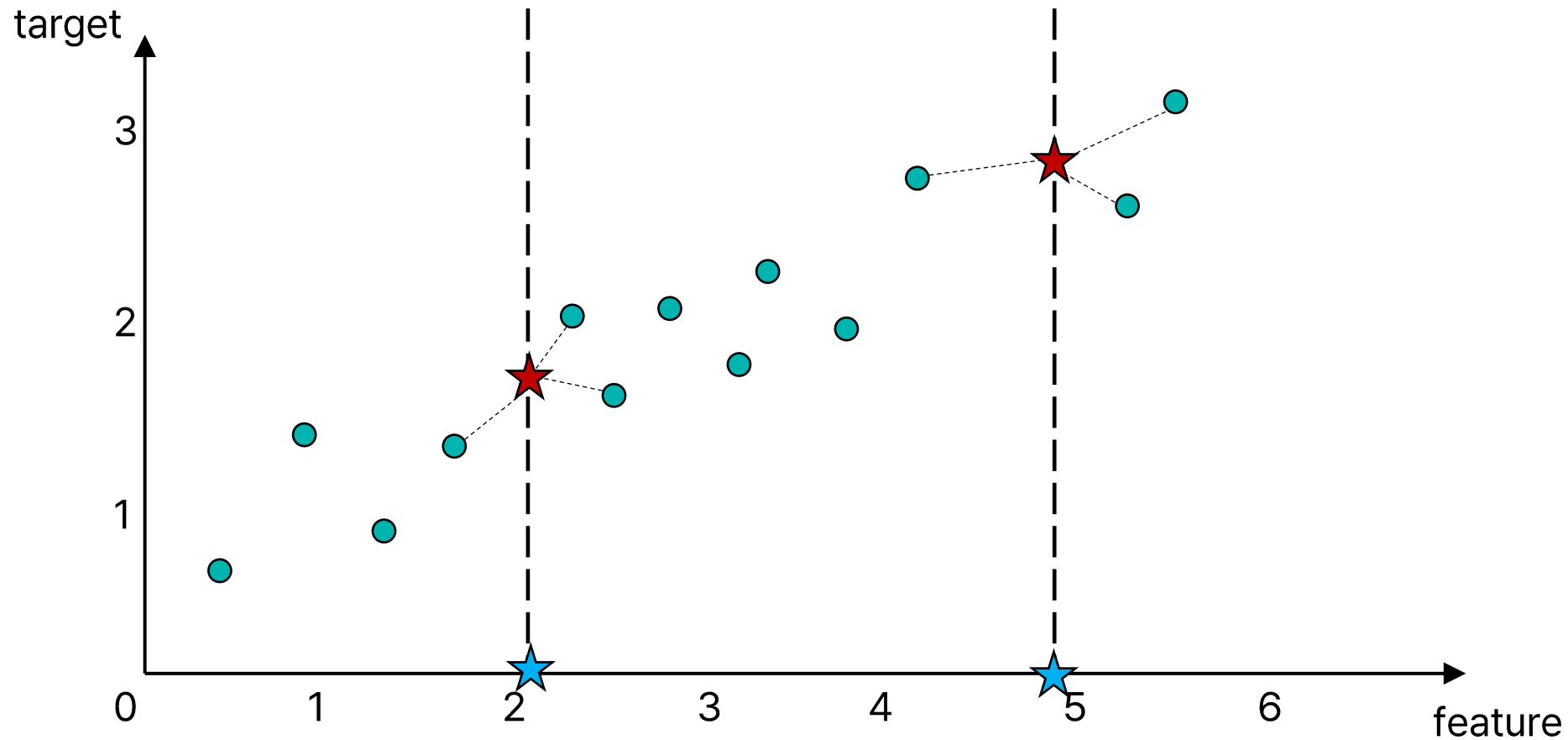
Section 2-5. k 최근접 이웃 회귀

k 최근접 이웃 알고리즘

● training data/target

★ test prediction

★ test data



k 최근접 이웃 알고리즘

```
[18]: import pandas as pd
df = pd.read_csv("../data/house_prices.csv")
```

```
[19]: df
```

[19]:

	CRIM	ZN	INDUS	CHAS	NOX	RM	AGE	DIS	RAD	TAX	PTRATIO	B	LSTAT	MEDV
0	0.00632	18.0	2.31	0	0.538	6.575	65.2	4.0900	1	296	15.3	396.90	4.98	24.0
1	0.02731	0.0	7.07	0	0.469	6.421	78.9	4.9671	2	242	17.8	396.90	9.14	21.6
2	0.02729	0.0	7.07	0	0.469	7.185	61.1	4.9671	2	242	17.8	392.83	4.03	34.7
3	0.03237	0.0	2.18	0	0.458	6.998	45.8	6.0622	3	222	18.7	394.63	2.94	33.4
4	0.06905	0.0	2.18	0	0.458	7.147	54.2	6.0622	3	222	18.7	396.90	5.33	36.2
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
501	0.06263	0.0	11.93	0	0.573	6.593	69.1	2.4786	1	273	21.0	391.99	9.67	22.4
502	0.04527	0.0	11.93	0	0.573	6.120	76.7	2.2875	1	273	21.0	396.90	9.08	20.6
503	0.06076	0.0	11.93	0	0.573	6.976	91.0	2.1675	1	273	21.0	396.90	5.64	23.9
504	0.10959	0.0	11.93	0	0.573	6.794	89.3	2.3889	1	273	21.0	393.45	6.48	22.0
505	0.04741	0.0	11.93	0	0.573	6.030	80.8	2.5050	1	273	21.0	396.90	7.88	11.9

506 rows × 14 columns

k 최근접 이웃 알고리즘

```
[20]: df['MEDV'].unique()
```



```
[20]: array([24. , 21.6, 34.7, 33.4, 36.2, 28.7, 22.9, 27.1, 16.5, 18.9, 15. ,
          21.7, 20.4, 18.2, 19.9, 23.1, 17.5, 20.2, 13.6, 19.6, 15.2, 14.5,
          15.6, 13.9, 16.6, 14.8, 18.4, 21. , 12.7, 13.2, 13.1, 13.5, 20. ,
          24.7, 30.8, 34.9, 26.6, 25.3, 21.2, 19.3, 14.4, 19.4, 19.7, 20.5,
          25. , 23.4, 35.4, 31.6, 23.3, 18.7, 16. , 22.2, 33. , 23.5, 22. ,
          17.4, 20.9, 24.2, 22.8, 24.1, 21.4, 20.8, 20.3, 28. , 23.9, 24.8,
          22.5, 23.6, 22.6, 20.6, 28.4, 38.7, 43.8, 33.2, 27.5, 26.5, 18.6,
          20.1, 19.5, 19.8, 18.8, 18.5, 18.3, 19.2, 17.3, 15.7, 16.2, 18. ,
          14.3, 23. , 18.1, 17.1, 13.3, 17.8, 14. , 13.4, 11.8, 13.8, 14.6,
          15.4, 21.5, 15.3, 17. , 41.3, 24.3, 27. , 50. , 22.7, 23.8, 22.3,
          19.1, 29.4, 23.2, 24.6, 29.9, 37.2, 39.8, 37.9, 32.5, 26.4, 29.6,
          32. , 29.8, 37. , 30.5, 36.4, 31.1, 29.1, 33.3, 30.3, 34.6, 32.9,
          42.3, 48.5, 24.4, 22.4, 28.1, 23.7, 26.7, 30.1, 44.8, 37.6, 46.7,
          31.5, 31.7, 41.7, 48.3, 29. , 25.1, 17.6, 24.5, 26.2, 42.8, 21.9,
          44. , 36. , 33.8, 43.1, 48.8, 31. , 36.5, 30.7, 43.5, 20.7, 21.1,
          25.2, 35.2, 32.4, 33.1, 35.1, 45.4, 46. , 32.2, 28.5, 37.3, 27.9,
          28.6, 36.1, 28.2, 16.1, 22.1, 19. , 32.7, 31.2, 17.2, 16.8, 10.2,
          10.4, 10.9, 11.3, 12.3,  8.8,  7.2, 10.5,  7.4, 11.5, 15.1,  9.7,
          12.5,  8.5,  5. ,  6.3,  5.6, 12.1,  8.3, 11.9, 17.9, 16.3,  7. ,
           7.5,  8.4, 16.7, 14.2, 11.7, 11. ,  9.5, 14.1,  9.6,  8.7, 12.8,
          10.8, 14.9, 12.6, 13. , 16.4, 17.7, 12. , 21.8,  8.1])
```

Section

k최근접이웃회귀

```
[21]: df.columns

[21]: Index(['CRIM', 'ZN', 'INDUS', 'CHAS', 'NOX', 'RM', 'AGE', 'DIS', 'RAD', 'TAX',
          'PTRATIO', 'B', 'LSTAT'],
          dtype='object')

[22]: features = ['CRIM', 'ZN', 'INDUS', 'CHAS', 'NOX', 'RM', 'AGE', 'DIS', 'RAD', 'TAX', 'PTRATIO', 'B', 'LSTAT']

X = df[features]
y = df['MEDV']

[23]: X

[23]:
```

	CRIM	ZN	INDUS	CHAS	NOX	RM	AGE	DIS	RAD	TAX	PTRATIO	B	LSTAT
0	0.00632	18.0	2.31	0	0.538	6.575	65.2	4.0900	1	296	15.3	396.90	4.98
1	0.02731	0.0	7.07	0	0.469	6.421	78.9	4.9671	2	242	17.8	396.90	9.14
2	0.02729	0.0	7.07	0	0.469	7.185	61.1	4.9671	2	242	17.8	392.83	4.03
3	0.03237	0.0	2.18	0	0.458	6.998	45.8	6.0622	3	222	18.7	394.63	2.94
4	0.06905	0.0	2.18	0	0.458	7.147	54.2	6.0622	3	222	18.7	396.90	5.33
...	...	...	...	...	...	...	...	...	...	...	...	...	...
501	0.06263	0.0	11.93	0	0.573	6.593	69.1	2.4786	1	273	21.0	391.99	9.67
502	0.04527	0.0	11.93	0	0.573	6.120	76.7	2.2875	1	273	21.0	396.90	9.08
503	0.06076	0.0	11.93	0	0.573	6.976	91.0	2.1675	1	273	21.0	396.90	5.64
504	0.10959	0.0	11.93	0	0.573	6.794	89.3	2.3889	1	273	21.0	393.45	6.48
505	0.04741	0.0	11.93	0	0.573	6.030	80.8	2.5050	1	273	21.0	396.90	7.88

506 rows × 13 columns

k 최근접 이웃 알고리즘

```
[24]: # 트레이닝/테스트 데이터 분할
      from sklearn.model_selection import train_test_split
      X_tn, X_te, y_tn, y_te=train_test_split(X,y,random_state=0)
```

```
[25]: #데이터 표준화
      from sklearn.preprocessing import StandardScaler
      std_scale = StandardScaler()
      std_scale.fit(X_tn)
      X_tn_std = std_scale.transform(X_tn)
      X_te_std = std_scale.transform(X_te)
```

k 최근접 이웃 알고리즘

```
# 학습
from sklearn.neighbors import KNeighborsRegressor
knn_model = KNeighborsRegressor(n_neighbors=2)
knn_model.fit(X_tn_std, y_tn)
```

▼ KNeighborsRegressor ⓘ ?

▶ Parameters

```
# 예측
y_pred = knn_model.predict(X_te_std)
print(y_pred)
```

```
[22.   17.6  23.3  10.35 20.55 19.3  20.9  18.75 23.15 14.15  7.95 10.8
 13.6  11.05 38.5  26.8  22.2  32.45 25.8  22.55 24.05 20.05 19.7  28.4
 21.85 21.9  16.55 17.6  44.9  18.55 14.4  18.6  19.9  21.35 27.35 20.25
 11.8  36.35 15.1  16.05 25.15 21.85 20.25 19.95 21.65 24.3  24.4  14.35
 16.2  23.75 21.15 23.15 23.3  27.9  17.9  20.7  19.55 17.45 13.6  19.65
 22.5  21.45 30.35 24.8  20.25 33.45 14.6  18.9  12.9  24.85 21.95 20.55
 23.3  30.45 27.4   8.7  39.65 24.05 29.1  17.15 22.1  18.6  16.75 39.65
 39.65 24.8  25.8  12.6  20.55 14.3  19.15 10.65 22.7  22.25 22.2  23.9
 12.   23.8  13.85 18.6  25.   19.75 26.3  23.1  22.1  18.9  12.3  17.95
 22.2  22.85 36.15 15.55 18.9  18.8  16.65 21.55  7.95 19.4  10.75 39.6
 36.7  12.05 19.55 19.3  20.5  20.95 32.15]
```

```
# 모형 평가-MSE
```

```
from sklearn.metrics import mean_squared_error
mse = mean_squared_error(y_te, y_pred)
print(mse)
```

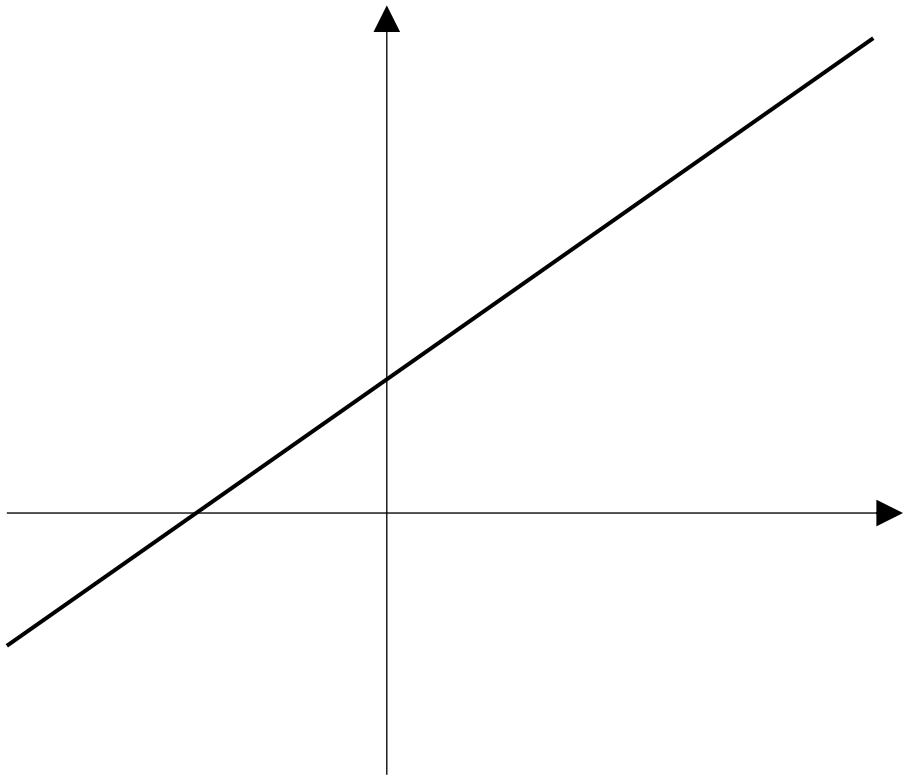
```
26.36383858267716
```

AI/BigData 인텐시브 과정

Section 3. 선형 회귀 분석

Section 3-1. 선형 회귀 분석의 개념

선형 회귀 분석

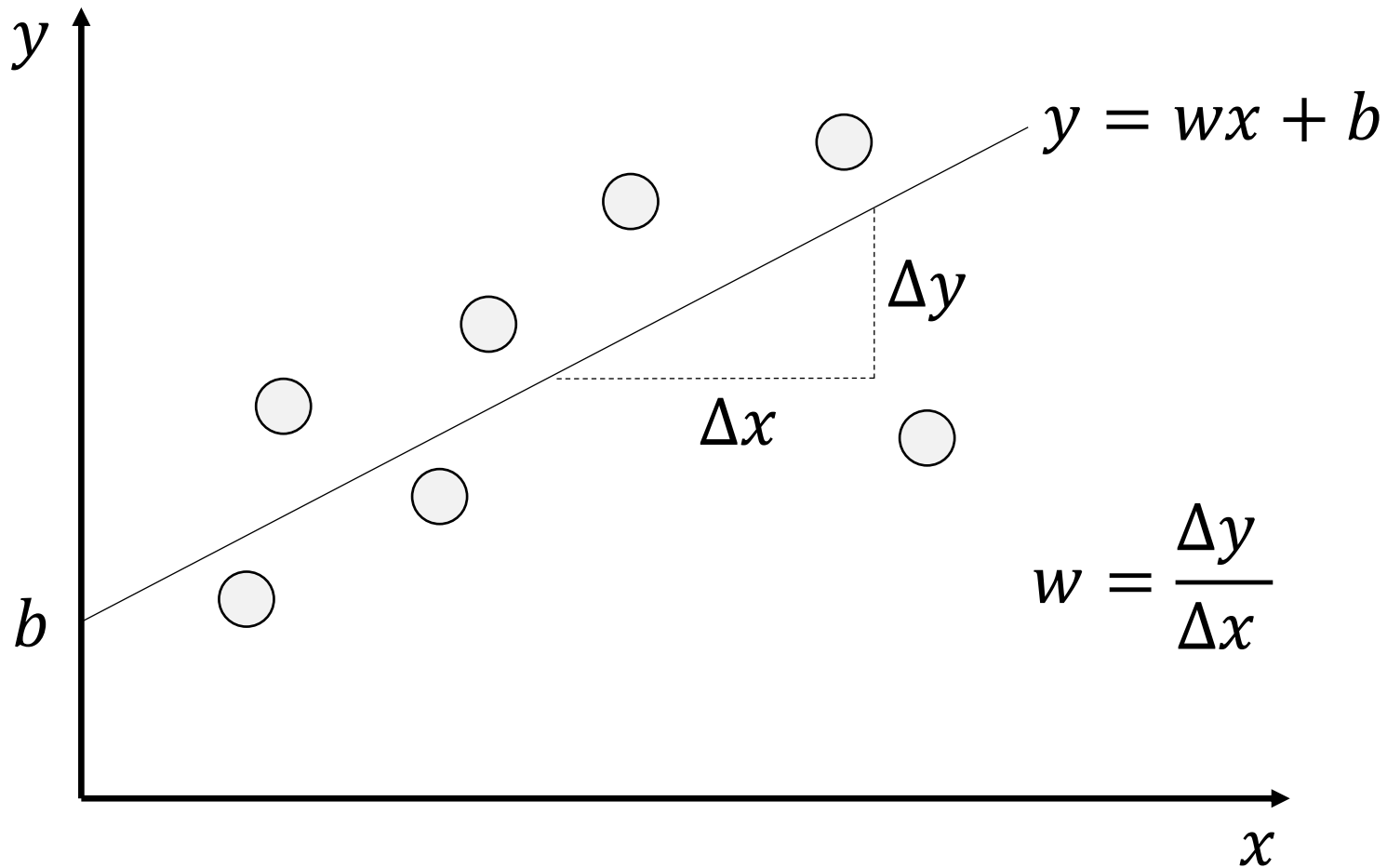


$$y = ax + b$$

↑
기울기

↑
y절편

선형 회귀 분석



$$y = wx + b$$

기울기 y절편

선형 회귀 분석

$$f(\mathbf{x}_i) = \hat{y}_i = w_1 x_1 + w_2 x_2 + \cdots + w_p x_p + b$$

$$f(\mathbf{x}_i) = \hat{y}_i = \mathbf{w}^T \mathbf{x}_i + b$$

선형 회귀 분석

첫번째 피처에 대한 가중치

첫번째 피처 스칼라
스칼라

p번째 피처

$$f(\mathbf{x}_i) = \hat{y}_i = w_1 x_1 + w_2 x_2 + \cdots + w_p x_p + b$$

$$f(\mathbf{x}_i) = \hat{y}_i = \mathbf{w}^T \mathbf{x}_i + b$$

$$\mathbf{w} = \begin{pmatrix} w_1 \\ w_2 \\ \vdots \\ w_p \end{pmatrix} \quad \mathbf{x} = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_p \end{pmatrix}$$

$$\mathbf{w}^T \mathbf{x} = (w_1 \quad w_2 \quad \cdots \quad w_p) \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_p \end{pmatrix}$$

w를 구하는 방법

$$y_i = \hat{y}_i + e_i, \quad i = 1, 2, \dots, n$$

실제값 예측값 오차

$$\text{오차 제곱합} = \sum_{i=1}^n e_i^2 = \sum_{i=1}^n [y_i - f(\mathbf{x}_i)]^2 = \sum_{i=1}^n (y_i - \mathbf{w}^T \mathbf{x}_i)^2$$

선형 회귀 분석

x_1	x_2	y	$\hat{y}$	$e = y - \hat{y}$
2	3	5.2	5.1	0.1
1	2	4.7	4.8	-0.1
4	5	3.2	3.4	-0.2
1	3	2.1	2.5	-0.4
2	2	1.9	1.2	0.7

$$y_i = \hat{y}_i + e_i, \quad i = 1, 2, \dots, n$$

$$\text{오차 제곱합} = \sum_{i=1}^n e_i^2 = \sum_{i=1}^n [y_i - f(\mathbf{x}_i)]^2 = \sum_{i=1}^n (y_i - \mathbf{w}^T \mathbf{x}_i)^2$$

목적: 오차 제곱합을 최소로 하는 $\mathbf{w}$ 구하기

선형 회귀 분석

X

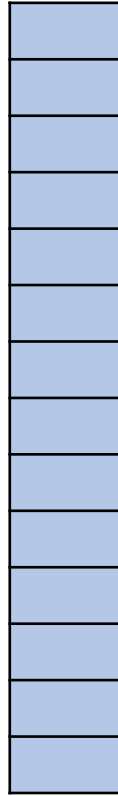
독립 변수, 피처

y

종속 변수, 타겟

선형 회귀 분석

X



독립 변수, 피처

y



종속 변수, 타겟

회귀 분석의 가정

(1) 변수 x 와 y 사이에는 다음과 같은 관련성이 존재한다.

$$E(y) = \beta_1 x + \beta_0$$

$$E(y) = wx + b$$

(2) 피쳐 x 값이 주어졌을 때, 타겟 y 는 정규 분포를 따른다.

(3) 이 때, y 의 평균은 $E(y) = wx + b$ 로 피쳐 x 값에 따라 변하나 분산은 x 값에 관계 없이 일정하다.

(4) 피쳐 x 는 오차 없이 측정할 수 있는 수학 변수(mathematical variable)이며, 타겟 y 는 측정 오차를 수반하는 확률 변수(random variable)이다.

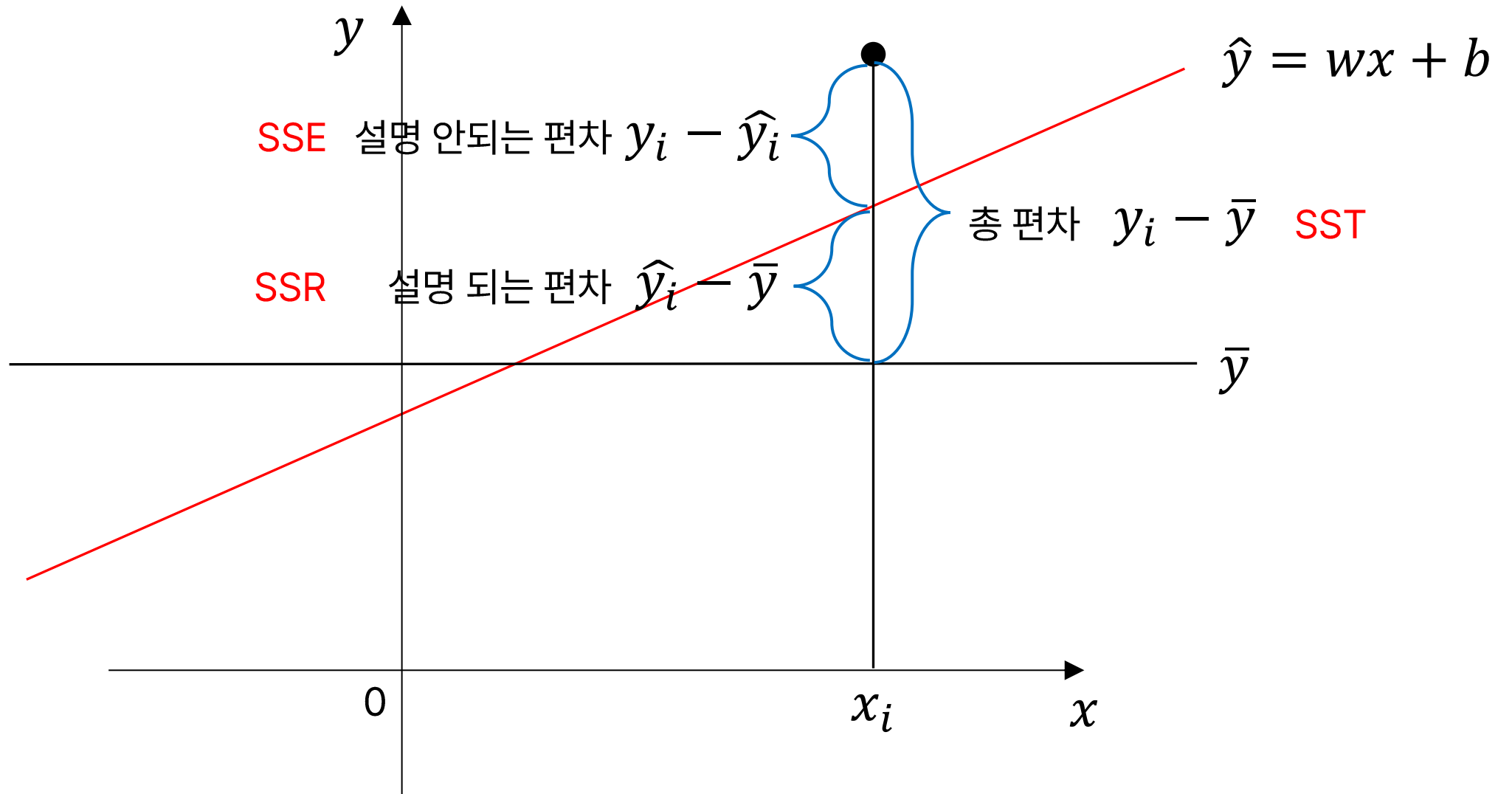
(5) 타겟 y 의 오차들은 서로 독립이다.

최소 제곱 추정량(least square estimator)

$$S = \sum_{i=1}^n \epsilon_i^2 = \sum_{i=1}^n (y_i - wx - b)^2$$

오차 제곱합을 최소로 하는 w, b 구하기

선형 회귀 분석



선형 회귀 분석

$$SST = SSE + SSR$$

총 변동

회귀 모형에 의해

회귀 모형에 의해

설명 안되는 변동

설명 되는 변동

$$\sum_{i=1}^n (y_i - \bar{y})^2 = \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \sum_{i=1}^n (\hat{y}_i - \bar{y})^2$$

결정 계수

$$r^2 = \frac{SSR}{SST} = 1 - \frac{SSE}{SST}$$

$$0 \leq r^2 \leq 1$$

이 범위라고 흔히 알려져 있지만 음수값도 나올 수 있음

전체 변동 중 회귀 모형으로 설명가능한 변동의 비율

결정 계수

$$r^2 = \frac{SSR}{SST} = 1 - \frac{SSE}{SST}$$

$$-\infty \leq r^2 \leq 1$$

어떻게 음수 값이 나올까요? 예를 들어 봅시다.

전체 변동 중 회귀 모형으로 설명가능한 변동의 비율

r 제곱값이 음수가 나오는 경우

$$R^2 = 1 - \frac{SSE}{SST}$$

y	$\hat{y}$
1	5
2	4
3	3
4	2
5	1

$$SST = \sum_{i=1}^n (y_i - \bar{y})^2 = (1 - 3)^2 + (2 - 3)^2 + (3 - 3)^2 + (4 - 3)^2 + (5 - 3)^2$$

$$= 4 + 1 + 0 + 1 + 4 = 10$$

$$SSE = \sum_{i=1}^n (y_i - \hat{y}_i)^2 = (1 - 5)^2 + (2 - 4)^2 + (3 - 3)^2 + (4 - 2)^2 + (5 - 1)^2$$

$$= 16 + 4 + 0 + 4 + 16 = 40$$

$$R^2 = 1 - \frac{SSE}{SST} = 1 - \frac{40}{10} = 1 - 4 = -3$$

평균으로 예측하는 것보다도 못할 때

$$\bar{y} = 3$$

다중공선성(multicollinearity)

 X

독립 변수, 피쳐

어떤 독립 변수가 다른 독립 변수들의 선형 조합으로 표현될 수 있는 경우

$$\text{EX) } x_1 = x_2 + x_3$$

차원 축소

$$y = w_1x_1 + w_2x_2 + w_3x_3 + b$$



노트북
판매량



디자인



전체 성능



cpu 성능

차원 축소

$$y = w_1x_1 + w_2x_2 + w_3x_3 + b$$



노트북
판매량

디자인

전체 성능

cpu 성능

$$\text{Corr}(x_1, x_2)$$

$$\text{Corr}(x_1, x_3)$$

$$\text{Corr}(x_2, x_3)$$

카이제곱 검정

Section



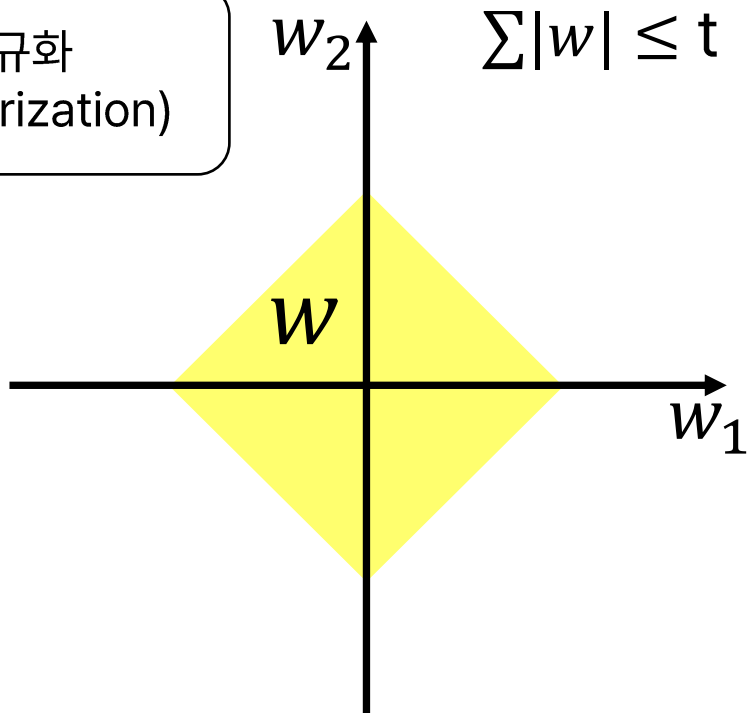
[illegible]

Regularization

$$y = w_1x_1 + w_2x_2 + b$$

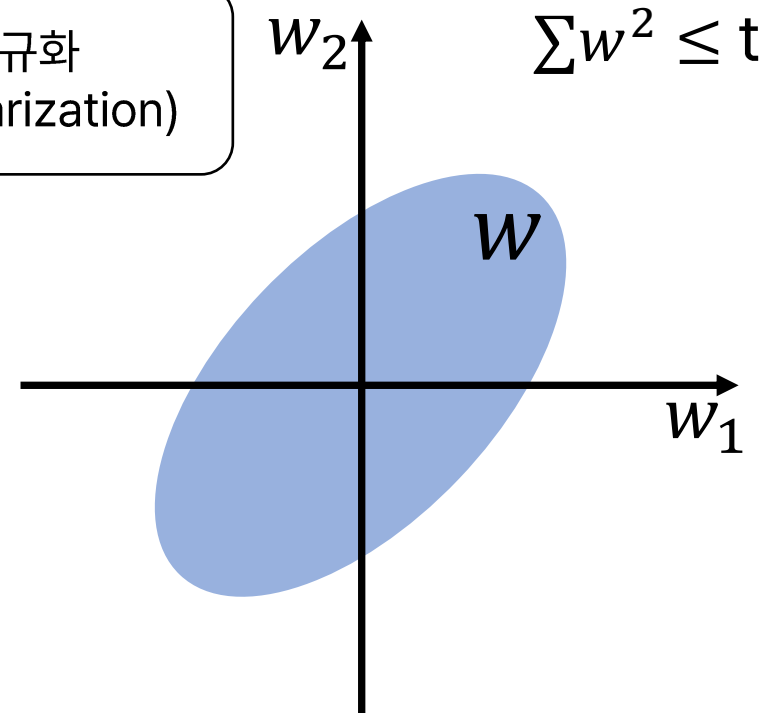
Lasso Regression

L1 정규화
(L1 regularization)

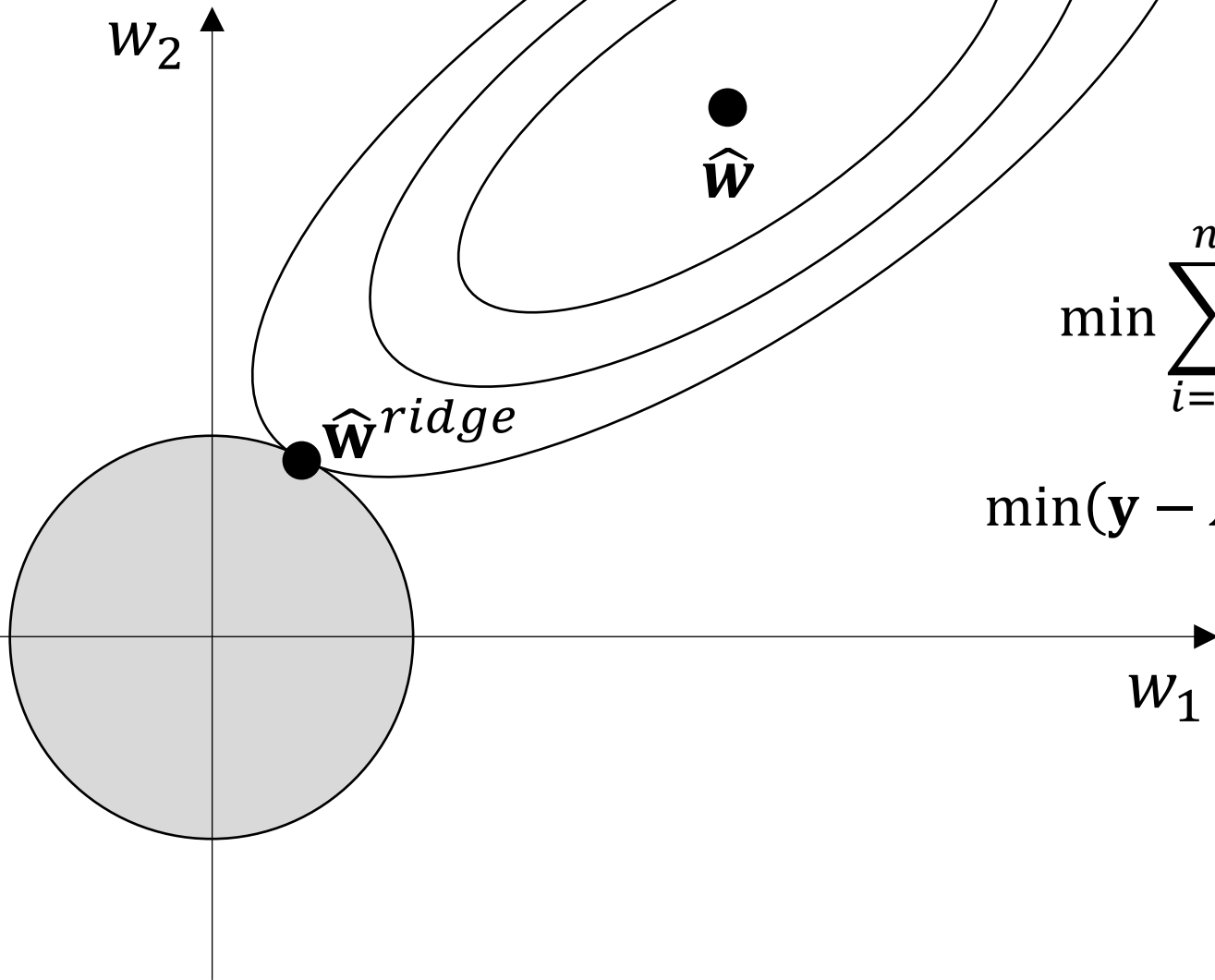


Ridge Regression

L2 정규화
(L2 regularization)



릿지 회귀 분석



$$\min \sum_{i=1}^n (y_i - \mathbf{w}^T \mathbf{x}_i)^2,$$

$$s.t. \sum_{i=1}^p w_i^2 \leq t$$

$$\min (\mathbf{y} - \mathbf{X}\mathbf{w})^T (\mathbf{y} - \mathbf{X}\mathbf{w}),$$

$$s.t. \|\mathbf{w}\|_2^2 \leq t$$

릿지 회귀 분석 - 라그랑주 프리멀 함수로 표현

$$L_p = (\mathbf{y} - \mathbf{X}\mathbf{w})^T(\mathbf{y} - \mathbf{X}\mathbf{w}) + \lambda(\|\mathbf{w}\|_2^2 - t)$$

$$\hat{\mathbf{w}}^{ridge} = \underset{\mathbf{w}}{\operatorname{argmin}} \{ (\mathbf{y} - \mathbf{X}\mathbf{w})^T(\mathbf{y} - \mathbf{X}\mathbf{w}) + \lambda(\|\mathbf{w}\|_2^2 - t) \}$$

릿지 회귀 분석

$$\frac{\partial L_p}{\partial \mathbf{w}} = -2\mathbf{X}^T(\mathbf{y} - \mathbf{X}\mathbf{w}) + 2\lambda\mathbf{w} = 0$$

$$\Leftrightarrow -2\mathbf{X}^T\mathbf{y} + 2\mathbf{X}^T\mathbf{X}\mathbf{w} + 2\lambda\mathbf{w} = 0$$

$$\Leftrightarrow (\mathbf{X}^T\mathbf{X} + \lambda\mathbf{I}_p)\mathbf{w} = \mathbf{X}^T\mathbf{y}$$

$$\hat{\mathbf{w}}^{ridge} = (\mathbf{X}^T\mathbf{X} + \lambda\mathbf{I}_p)^{-1}\mathbf{X}^T\mathbf{y}$$

람다의 역할

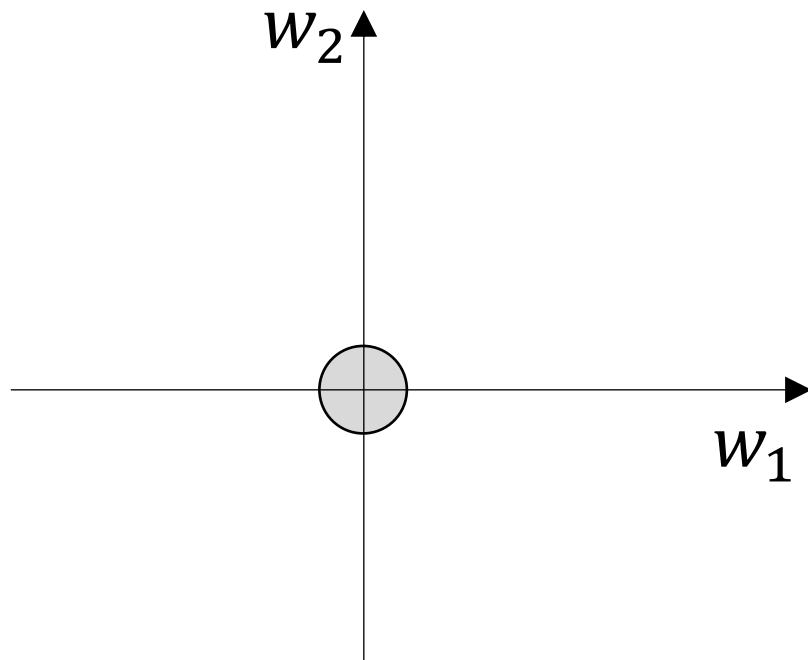
- λ 는 계수(coefficient)의 사이즈를 조절
- λ 는 정규식의 크기(amount)를 조절
- λ 가 0에 가까워질수록, 최소 제곱 추정량(least squared estimator)에 가까워짐
- λ 가 무한대에 가까워질수록 릿지 해는 0에 가까워집니다. 즉, 상수항(intercept)만 남은 모형에 가까워짐

제약식 가중치

$$\lambda \sum_{i=1}^p w_i^2 \leq t \Leftrightarrow \sum_{i=1}^p w_i^2 \leq \frac{t}{\lambda}$$

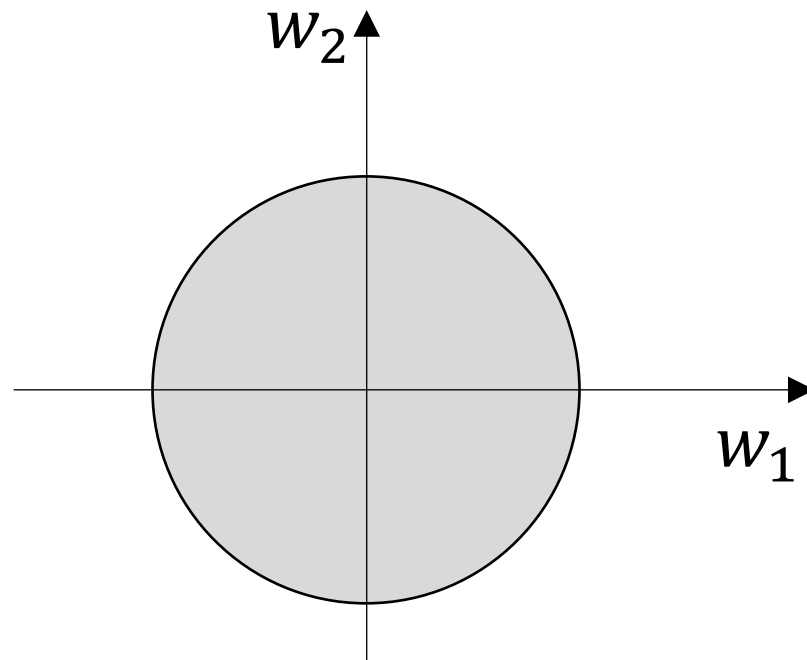
람다가 커지면?

-> 도형이 작아짐 -> 제약식이 강해짐

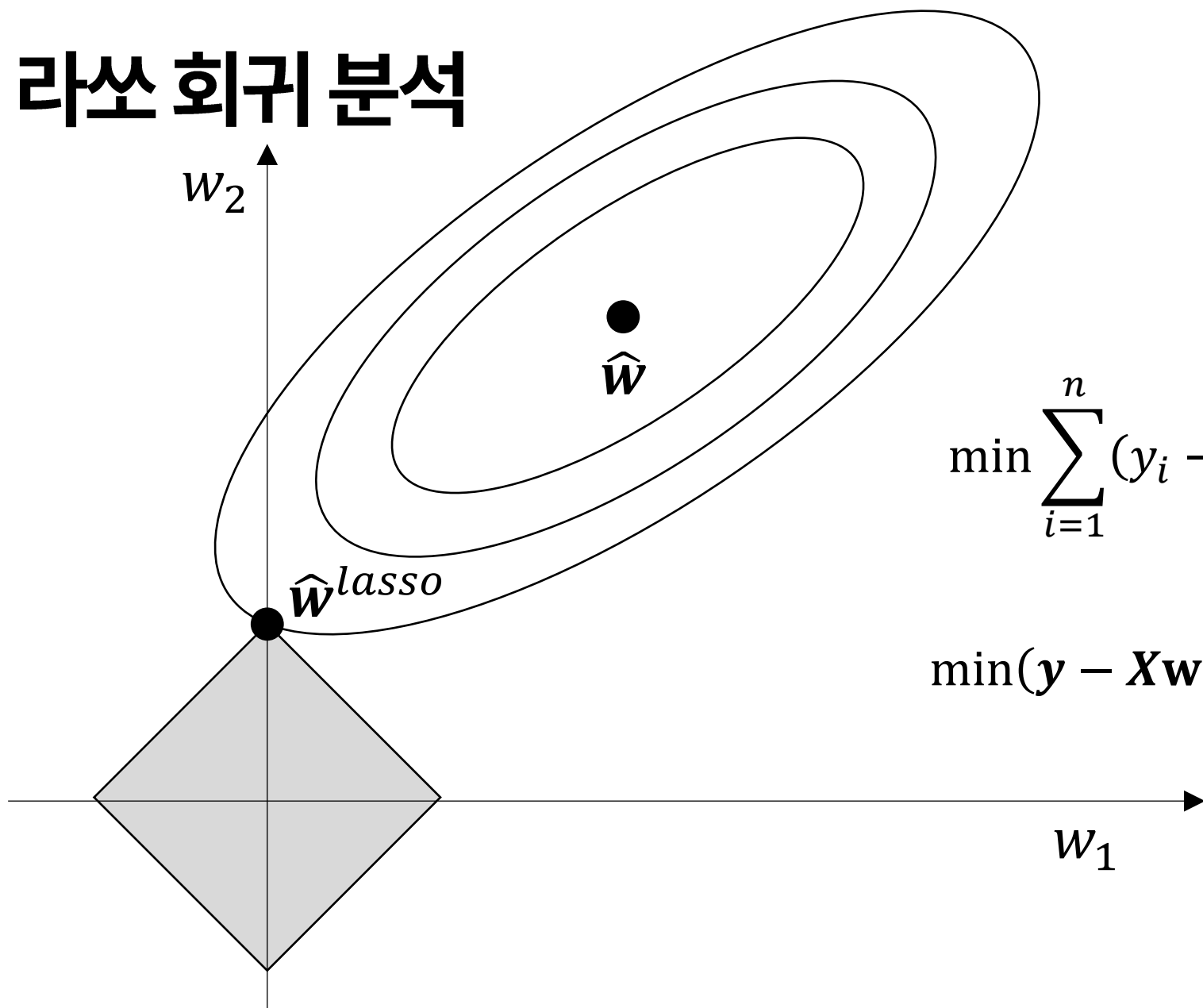


람다가 작아지면?

-> 도형이 커짐 -> 제약식이 약해짐



라쏘 회귀 분석



$$\min \sum_{i=1}^n (y_i - \mathbf{w}^T \mathbf{x}_i)^2, \quad s.t. \sum_{i=1}^d |\mathbf{w}_i| \leq t$$

$$\min (\mathbf{y} - \mathbf{X}\mathbf{w})^T (\mathbf{y} - \mathbf{X}\mathbf{w}), \quad s.t. |\mathbf{w}| \leq t$$

라쏘 회귀 분석 - 라그랑주 프리멀 함수로 표현

$$L_P = (\mathbf{y} - X\mathbf{w})^T(\mathbf{y} - X\mathbf{w}) + \lambda(|\mathbf{w}| - t)$$

$$\hat{\mathbf{w}}^{lasso} = \underset{\mathbf{w}}{\operatorname{argmin}}\{(\mathbf{y} - X\mathbf{w})^T(\mathbf{y} - X\mathbf{w}) + \lambda(|\mathbf{w}| - t)\}$$

라쏘 회귀 분석 - 추정량

라쏘 추정량은 확실한 형태(closed form solution)가 존재하지 않음

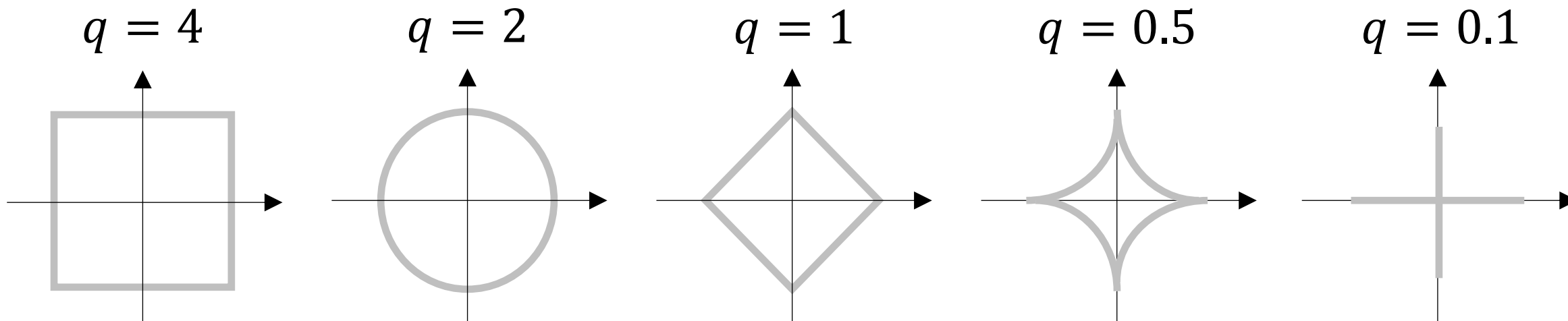
라쏘 추정량은 closed form이 존재하지 않기 때문에,

소프트웨어 기반으로 경사하강법과 같은 numerical optimization을 사용하여 구함

제약식의 일반화된 표현

$$\min \sum_{i=1}^n (y_i - \mathbf{w}^T \mathbf{x}_i)^2, \quad s.t. \sum_{i=1}^d |\mathbf{w}_i|^q \leq t$$

$$\tilde{\mathbf{w}} = \underset{\mathbf{w}}{\operatorname{argmin}} \{ (\mathbf{y} - \mathbf{X}\mathbf{w})^T (\mathbf{y} - \mathbf{X}\mathbf{w}) + \lambda (|\mathbf{w}|^q - t) \}$$



엘라스틱 넷(Elastic net)

엘라스틱 넷은 릿지 회귀 분석과 라쏘 회귀 분석을 합쳐놓은 형태
L1 제약식과 L2 제약식을 동시에 적용

$$\min \sum_{i=1}^n (y_i - \mathbf{w}^T \mathbf{x}_i)^2, \quad s.t. \sum_{i=1}^d (\alpha |w_i| + (1 - \alpha) w_i^2) \leq t$$

$$\min (\mathbf{y} - \mathbf{X}\mathbf{w})^T (\mathbf{y} - \mathbf{X}\mathbf{w}), \quad s.t. (\alpha \|\mathbf{w}\|_1 + (1 - \alpha) \|\mathbf{w}\|_2^2) \leq t$$

엘라스틱 넷(Elastic net)

$$L_p = (\mathbf{y} - \mathbf{X}\mathbf{w})^T(\mathbf{y} - \mathbf{X}\mathbf{w}) + \lambda(\alpha|\mathbf{w}| + (1 - \alpha)\|\mathbf{w}\|_2^2 - t)$$

$$\hat{\mathbf{w}}^{elasticnet} = \underset{\mathbf{w}}{\operatorname{argmin}}\{(\mathbf{y} - \mathbf{X}\mathbf{w})^T(\mathbf{y} - \mathbf{X}\mathbf{w}) + \lambda(\alpha|\mathbf{w}| + (1 - \alpha)\|\mathbf{w}\|_2^2 - t)\}$$

AI/BigData 인텐시브 과정

Section 3. 선형 회귀 분석

Section 3-2. 선형 회귀 분석의 실습

선형 회귀 분석

```
In [1]: 1 import pandas as pd
        2 df = pd.read_csv("../data/house_prices.csv")
```

```
In [2]: 1 df
```

Out[2]:

	CRIM	ZN	INDUS	CHAS	NOX	RM	AGE	DIS	RAD	TAX	PTRATIO	B	LSTAT	MEDV
0	0.00632	18.0	2.31	0	0.538	6.575	65.2	4.0900	1	296	15.3	396.90	4.98	24.0
1	0.02731	0.0	7.07	0	0.469	6.421	78.9	4.9671	2	242	17.8	396.90	9.14	21.6
2	0.02729	0.0	7.07	0	0.469	7.185	61.1	4.9671	2	242	17.8	392.83	4.03	34.7
3	0.03237	0.0	2.18	0	0.458	6.998	45.8	6.0622	3	222	18.7	394.63	2.94	33.4
4	0.06905	0.0	2.18	0	0.458	7.147	54.2	6.0622	3	222	18.7	396.90	5.33	36.2
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
501	0.06263	0.0	11.93	0	0.573	6.593	69.1	2.4786	1	273	21.0	391.99	9.67	22.4
502	0.04527	0.0	11.93	0	0.573	6.120	76.7	2.2875	1	273	21.0	396.90	9.08	20.6
503	0.06076	0.0	11.93	0	0.573	6.976	91.0	2.1675	1	273	21.0	396.90	5.64	23.9
504	0.10959	0.0	11.93	0	0.573	6.794	89.3	2.3889	1	273	21.0	393.45	6.48	22.0
505	0.04741	0.0	11.93	0	0.573	6.030	80.8	2.5050	1	273	21.0	396.90	7.88	11.9

506 rows × 14 columns

선형 회귀 분석

```
In [5]: ▶ 1 features = ['CRIM', 'ZN', 'INDUS', 'CHAS', 'NOX', 'RM', 'AGE', 'DIS', 'RAD', 'TAX', 'PTRATIO', 'B', 'LSTAT']  
2  
3 X = df[features]  
4 y = df['MEDV']
```

```
In [6]: ▶ 1 # 트레이닝/테스트 데이터 분할  
2 from sklearn.model_selection import train_test_split  
3 X_tn, X_te, y_tn, y_te=train_test_split(X,y,random_state=0)
```

```
In [7]: ▶ 1 #데이터 표준화  
2 from sklearn.preprocessing import StandardScaler  
3 std_scale = StandardScaler()  
4 std_scale.fit(X_tn)  
5 X_tn_std = std_scale.transform(X_tn)  
6 X_te_std = std_scale.transform(X_te)
```


선형 회귀 분석

```
# 선형 회귀분석 학습
```

```
from sklearn.linear_model import LinearRegression
```

```
lr_model = LinearRegression()
```

```
lr_model.fit(X_tn_std, y_tn)
```

▼ LinearRegression ⓘ ?

▼ Parameters

fit_intercept	True
copy_X	True
tol	1e-06
n_jobs	None
positive	False

선형 회귀 분석

```
# 선형 회귀분석 모델 추정 계수 확인  
print(lr_model.coef_)  
print(lr_model.intercept_)
```

```
[-0.97100092  1.04667838 -0.04044753  0.59408776 -1.80876877  2.60991991  
 -0.19823317 -3.00216551  2.08021582 -1.93289037 -2.15743759  0.75199122  
 -3.59027047]  
22.608707124010557
```

$$y = w_1x_1 + w_2x_2 + w_3x_3 + \cdots + w_{13}x_{13} + b$$



$$y = -0.97x_1 + 1.04x_2 - 0.04x_3 + \cdots - 3.59x_{13} + 22.60$$

선형 회귀 분석

```
[9]: # 릿지 회귀분석(L2 제약식 적용)
      from sklearn.linear_model import Ridge
      ridge_model = Ridge(alpha=1)
      ridge_model.fit(X_tn_std, y_tn)
```

```
[9]: ▾ Ridge ⓘ ⓘ
      ▶ Parameters
```

```
[10]: # 라쏘 회귀분석(L1 제약식 적용)
       from sklearn.linear_model import Lasso
       lasso_model = Lasso(alpha=0.01)
       lasso_model.fit(X_tn_std, y_tn)
```

```
[10]: ▾ Lasso ⓘ ⓘ
       ▶ Parameters
```

```
[11]: # 엘라스틱넷
       from sklearn.linear_model import ElasticNet
       elastic_model = ElasticNet(alpha=0.01, l1_ratio=0.01)
       elastic_model.fit(X_tn_std, y_tn)
```

```
[11]: ▾ ElasticNet ⓘ ⓘ
       ▶ Parameters
```

선형 회귀 분석

```
[12]: # 예측
y_pred_lr = lr_model.predict(X_te_std)
y_pred_ridge = ridge_model.predict(X_te_std)
y_pred_lasso = lasso_model.predict(X_te_std)
y_pred_elastic = elastic_model.predict(X_te_std)
```

```
[14]: # 모형 평가-R제곱값
from sklearn.metrics import r2_score
print(r2_score(y_te, y_pred_lr))
print(r2_score(y_te, y_pred_ridge))
print(r2_score(y_te, y_pred_lasso))
print(r2_score(y_te, y_pred_elastic))
```

```
0.6354638433202133
0.6345884564889055
0.6343061000666704
0.6322273400977834
```

```
[15]: # 모형 평가-MSE
from sklearn.metrics import mean_squared_error
print(mean_squared_error(y_te, y_pred_lr))
print(mean_squared_error(y_te, y_pred_ridge))
print(mean_squared_error(y_te, y_pred_lasso))
print(mean_squared_error(y_te, y_pred_elastic))
```

```
29.782245092302333
29.853763334547597
29.876831576246808
30.046664219036877
```

AI/BigData 인텐시브 과정

Section 4. 로지스틱 회귀 분석

Section 4-1. 로지스틱 회귀 분석 개념

로지스틱 회귀 분석

	선형 회귀분석	로지스틱 회귀분석
타겟 형태	연속형	범주형, 연속형
타겟 범위	제한 없음	제한 있음

로지스틱 회귀 분석

$$z = wx + b$$



기울기



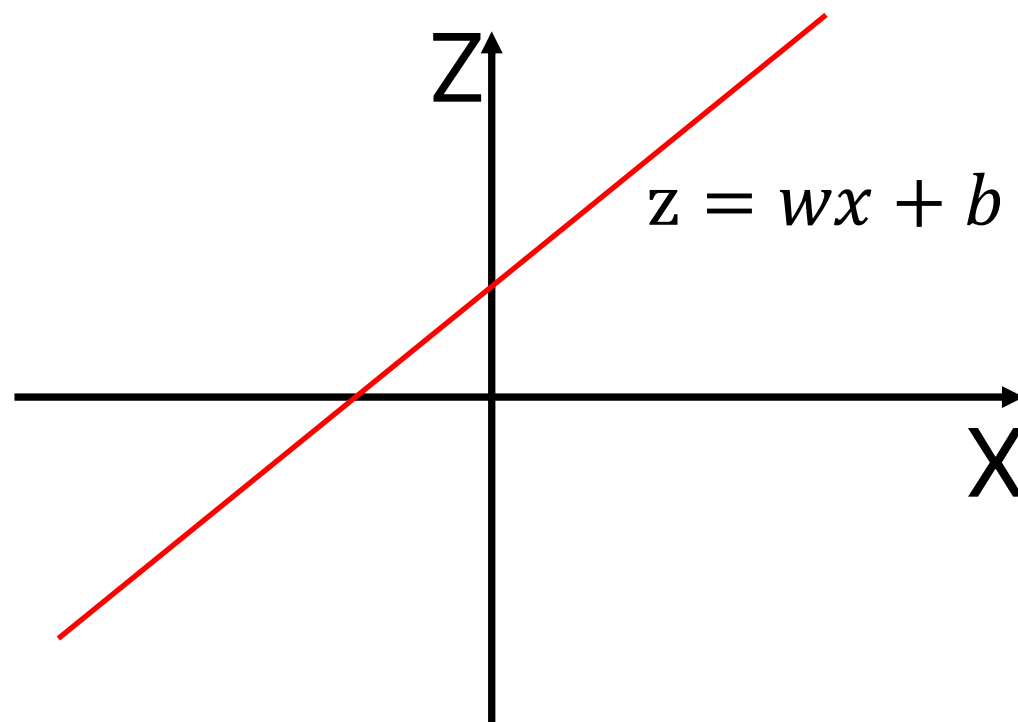
z 절편



$$-\infty < z < \infty$$



z에 제한이 있도록 변형 필요



로지스틱 회귀 분석

$$y = \frac{1}{1 + e^{-z}} = \frac{1}{1 + e^{-(wx+b)}}$$

z의 변형을 통해 만들어진 새로운 변수

시그모이드 함수

$$\log \left(\frac{y}{1-y} \right) = wx + b$$

AI/BigData 인텐시브 과정

Section 4. 로지스틱 회귀 분석

Section 4-1. 로지스틱 회귀 분석 실습

로지스틱 회귀 분석

```
In [1]: 1 import pandas as pd
        2 df = pd.read_csv("./data/wine_data.csv")
```

```
In [2]: 1 df
```

Out[2]:

Alcohol	Malic	Ash	Alcalinity	Magesium	Phenols	Flavanoids	Nonflavanoids	Proanthocyanins	Color	Hue	Dilution	Proline	class
14.23	1.71	2.43	15.6	127	2.80	3.06	0.28	2.29	5.64	1.04	3.92	1065	0
13.20	1.78	2.14	11.2	100	2.65	2.76	0.26	1.28	4.38	1.05	3.40	1050	0
13.16	2.36	2.67	18.6	101	2.80	3.24	0.30	2.81	5.68	1.03	3.17	1185	0
14.37	1.95	2.50	16.8	113	3.85	3.49	0.24	2.18	7.80	0.86	3.45	1480	0
13.24	2.59	2.87	21.0	118	2.80	2.69	0.39	1.82	4.32	1.04	2.93	735	0
...	...	...	...	...	...	...	...	...	...	...	...	...	...
13.71	5.65	2.45	20.5	95	1.68	0.61	0.52	1.06	7.70	0.64	1.74	740	2
13.40	3.91	2.48	23.0	102	1.80	0.75	0.43	1.41	7.30	0.70	1.56	750	2
13.27	4.28	2.26	20.0	120	1.59	0.69	0.43	1.35	10.20	0.59	1.56	835	2
13.17	2.59	2.37	20.0	120	1.65	0.68	0.53	1.46	9.30	0.60	1.62	840	2
14.13	4.10	2.74	24.5	96	2.05	0.76	0.56	1.35	9.20	0.61	1.60	560	2

rows × 14 columns

로지스틱 회귀 분석

```
In [3]: ▶ 1 features = ['Alcohol', 'Malic', 'Ash', 'Alcalinity', 'Magesium', 'Phenols', 'Flavanoids',  
2               'Nonflavanoids', 'Proanthocyanins', 'Color', 'Hue', 'Dilution', 'Proline']  
3  
4 X = df[features]  
5 y = df['class']
```

로지스틱 회귀 분석

In [4]: ▶

```
1 # 트레이닝/테스트 데이터 분할
2 from sklearn.model_selection import train_test_split
3 X_tn, X_te, y_tn, y_te=train_test_split(X,y,random_state=0)
```

In [5]: ▶

```
1 #데이터 표준화
2 from sklearn.preprocessing import StandardScaler
3 std_scale = StandardScaler()
4 std_scale.fit(X_tn)
5 X_tn_std = std_scale.transform(X_tn)
6 X_te_std = std_scale.transform(X_te)
```

로지스틱 회귀 분석

```
[6]: # 로지스틱 회귀분석(L2 제약식 적용)
from sklearn.linear_model import LogisticRegression
logistic_model = LogisticRegression(penalty='l2')
logistic_model.fit(X_tn_std, y_tn)
```

```
[6]: ▾ LogisticRegression ⓘ ?
```

```
▶ Parameters
```

```
[7]: # 로지스틱 회귀분석 모형(L2 제약식 적용) 추정 계수
print(logistic_model.coef_)
print(logistic_model.intercept_)
```

```
[[ 0.69901545  0.25725109  0.39788618 -0.76706283 -0.00437497  0.18428722
  0.7519187  -0.12523765  0.10350028  0.17553988  0.13496587  0.68116611
  0.98862011]
 [-0.81640371 -0.47221369 -0.74026158  0.5486431  -0.08498032  0.01394164
  0.18635383  0.1298404  0.32386105 -1.07230402  0.57757653  0.12740081
 -0.99263521]
 [ 0.11738827  0.21496261  0.3423754  0.21841973  0.08935529 -0.19822886
 -0.93827253 -0.00460275 -0.42736133  0.89676415 -0.7125424  -0.80856692
  0.0040151 ]
 [ 0.29711405  0.57453005 -0.8716441 ]
```

로지스틱 회귀 분석

[8]: # 예측

```
y_pred = logistic_model.predict(X_te_std)
print(y_pred)
```

```
[0] 2 1 0 1 1 0 2 1 1 2 2 0 1 2 1 0 0 1 0 1 0 0 1 1 1 1 1 2 0 0 1 0 0 0 2
    1 1 2 0 0 1 1 1]
```

[9]: # 확률값으로 예측

```
y_pred_proba = logistic_model.predict_proba(X_te_std)
print(y_pred_proba)
```

```
[[9.93402139e-01 4.88359790e-03 1.71426316e-03]
 [3.25964305e-04 2.76860324e-04 9.99397175e-01]
 [1.00515239e-02 9.89856845e-01 9.16311248e-05]
 [9.85658808e-01 1.14924783e-02 2.84871404e-03]
 [9.73276890e-02 8.94485108e-01 8.18720293e-03]
 [1.63034589e-01 8.35554073e-01 1.41133851e-03]
 [9.99185058e-01 2.81863941e-04 5.33078157e-04]
 [9.32776509e-04 6.78351531e-03 9.92283708e-01]
 [4.74344716e-03 9.94400029e-01 8.56523483e-04]
 [1.16449252e-03 9.96555584e-01 2.27992371e-03]
 [2.29832098e-02 1.70024450e-02 9.60014345e-01]
 [5.37966113e-04 2.05116751e-03 9.97410866e-01]
```

0일 확률

1일 확률

2일 확률

[9.93402139e-01	4.88359790e-03	1.71426316e-03]
[3.25964305e-04	2.76860324e-04	9.99397175e-01]
[1.00515239e-02	9.89856845e-01	9.16311248e-05]
[9.85658808e-01	1.14924783e-02	2.84871404e-03]
[9.73276890e-02	8.94485108e-01	8.18720293e-03]
[1.63034589e-01	8.35554073e-01	1.41133851e-03]
[9.99185058e-01	2.81863941e-04	5.33078157e-04]
[9.32776509e-04	6.78351531e-03	9.92283708e-01]
[4.74344716e-03	9.94400029e-01	8.56523483e-04]
[1.16449252e-03	9.96555584e-01	2.27992371e-03]
[2.29832098e-02	1.70024450e-02	9.60014345e-01]
[5.37966113e-04	2.05116751e-03	9.97410866e-01]
[9.99868867e-01	4.14562853e-05	8.96771895e-05]
[7.09646878e-02	9.28836492e-01	1.98820650e-04]
[3.45128597e-03	1.00259269e-03	9.95546121e-01]
[1.22433778e-04	9.99871471e-01	6.09484455e-06]
[9.95143149e-01	3.24958639e-03	1.60726501e-03]
[9.99969722e-01	5.36472664e-06	2.49137149e-05]
[5.82617126e-03	7.65631843e-01	2.28541986e-01]
[9.98989348e-01	9.22522694e-04	8.81295408e-05]

로지스틱 회귀 분석

```
[10]: # 정확도(accuracy)
      from sklearn.metrics import accuracy_score
      accuracy = accuracy_score(y_te, y_pred)
      print(accuracy)
```

1.0

```
[11]: # 정밀도(precision)
      from sklearn.metrics import precision_score
      precision = precision_score(y_te, y_pred, average='macro')
      print(precision)
```

1.0

```
[12]: # 리콜
      from sklearn.metrics import recall_score
      recall = recall_score(y_te, y_pred, average='macro')
      print(recall)
```

1.0

```
[13]: # F1 스코어
      from sklearn.metrics import f1_score
      f1 = f1_score(y_te, y_pred, average='macro')
      print(f1)
```

1.0

로지스틱 회귀 분석

```
[14]: # confusion matrix 확인  
from sklearn.metrics import confusion_matrix  
conf_matrix = confusion_matrix(y_te, y_pred)  
print(conf_matrix)
```

```
[[16  0  0]  
 [ 0 21  0]  
 [ 0  0  8]]
```

```
[15]: # 분류 레포트 확인  
from sklearn.metrics import classification_report  
class_report = classification_report(y_te, y_pred)  
print(class_report)
```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	16
1	1.00	1.00	1.00	21
2	1.00	1.00	1.00	8
accuracy			1.00	45
macro avg	1.00	1.00	1.00	45
weighted avg	1.00	1.00	1.00	45

감사합니다.

Q & A